

NEXUS-IDS: From structural to semantic explainability in flow-node graph neural networks for IoT intrusion detection

Maha Driss ^{a,b,*}, Safa Ben Atitallah ^{a,b}, Adel Ammar ^a, Wadii Boulila ^{a,b}

^a Robotics and Internet of Things Laboratory, Prince Sultan University Riyadh, 12435, Saudi Arabia

^b RIADI Laboratory, National School of Computer Science, University of Manouba, Manouba, 2010, Tunisia

ARTICLE INFO

Keywords:

Internet of things (IoT) security
Intrusion detection system (IDS)
Large language models (LLM)
Explainable AI (XAI)
Cybersecurity

ABSTRACT

The exponential growth of the Internet of Things (IoT) ecosystem demands accurate and explainable intrusion detection systems (IDS). This study introduces NEXUS-IDS, a framework for trustworthy IoT security analytics that integrates graph-based deep learning, Explainable AI (XAI), and Large Language Models (LLMs). Central to the framework is a Flow-Node Graph Neural Network (IDS-FN-GNN), which models network flows as nodes and communication relationships as edges to capture relational attack patterns. To enhance transparency, a dual-layer explainability mechanism is proposed that combines GraphLIME-based structural feature attributions with LLM-driven semantic explanations. This approach translates the model outputs into analyst-readable threat narratives supported by entropy-aware uncertainty calibration. Experiments on the Edge-IIoTSet and IoTID20 datasets yielded detection accuracies of 94.30% and 97.42%, respectively. At the default threshold, $\tau = 0.3$, entropy-based filtering forwards 19.33% of total incoming flows on the Edge-IIoTSet and 32.08% on IoTID20 directly to the semantic layer. Within this forwarded subset, the semantic cache resolves the majority of requests, yielding cache hit rates of 14.33% and 27.29% of the total incoming flows, respectively, thereby successfully reducing fresh LLM invocations to a minimal 5.00% and 4.78%. An automated LLM-as-a-judge assessment and a structured human expert evaluation across four architectures (82M–70B parameters) demonstrated high alignment. Human evaluation of a random sample verified that the foundation models achieved mean expert quality scores of up to 4.2/5.0. However, expert-flagged hallucination rates varied widely (0%–77%), indicating that perceived quality and factual reliability require independent assessment. Consequently, the LLM-driven semantic layer is positioned as an assistive research prototype. While the underlying detection and structural attributions remain robust, operationalizing these semantic explanations currently necessitates Human-in-the-Loop (HITL) oversight to ensure appropriate trust calibration.

1. Introduction

The Internet of Things (IoT) has experienced exponential growth over the last decade and has revolutionized modern digital ecosystems by interconnecting billions of heterogeneous devices across healthcare, industry, transportation, and smart cities [1]. Although this hyperconnectivity enables intelligent automation, it also expands the cyber-attack surface, exposing IoT infrastructure to large-scale attacks such as distributed denial-of-service (DDoS), data exfiltration, and malware propagation [2,3]. Conventional In-

* Corresponding author.

E-mail address: mdriss@psu.edu.sa (M. Driss).

<https://doi.org/10.1016/j.iot.2026.102010>

Received 6 January 2026; Received in revised form 4 June 2026; Accepted 22 June 2026

Available online 25 June 2026

2542-6605/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

intrusion Detection Systems (IDS) often fail to keep pace with these evolving threats owing to their limited scalability, poor adaptability to heterogeneous environments, and lack of transparency in their decision-making processes [4].

Recent advances in machine and deep learning have enabled sophisticated intrusion detection capabilities [5]. Nevertheless, the growing complexity of models poses significant trust and accountability issues, especially in mission-critical IoT applications. However, the black-box nature of deep neural networks makes it difficult for security analysts to understand, validate, and act on the detection results. Consequently, integrating Explainable AI (XAI) techniques into intrusion detection has become essential to improve transparency, trustworthiness, and compliance with emerging AI governance standards [6].

Moreover, as Large Language Models (LLMs) advance natural language reasoning, their capacity to generate contextual, human-readable explanations introduces a new dimension to security analytics. When properly aligned with structured explainability methods, such as GraphLIME or GNNExplainer, LLMs can bridge the semantic gap between low-level network events and high-level human interpretation [7]. However, deploying LLMs for security-critical applications introduces unique challenges, including computational overhead that is not suitable for real-time detection, hallucination risks that could mislead security analysts, and the need for uncertainty quantification to calibrate trust in generated explanations. Addressing these challenges requires novel architectural designs that balance the explanation quality with operational efficiency.

To address these challenges, the NEXUS-IDS (Natural EXplanations from strUctural to Semantic for IoT Intrusion Detection Systems) was introduced as an innovative framework that emphasizes a human-centric approach by integrating graph-based deep learning, XAI, and LLM-powered semantic interpretations. The primary objective of this study is to provide an integrated approach to transparent and actionable IoT intrusion detection. The proposed framework employs a Flow-Node Graph Neural Network (IDS-FN-GNN) model that captures spatial dependencies and relational patterns within IoT traffic by representing network flows as nodes and communication relationships as edges. Its dual-layer explainability combines structural reasoning via GraphLIME [8] with semantic reasoning using an LLM-powered interpreter. The semantic layer employs entropy-aware uncertainty calibration to flag low-confidence predictions, contextual feature enrichment to ground explanations in observable network behavior, and semantic caching to minimize computational overhead. This design translates model attributions into natural-language threat narratives, communicates prediction uncertainty transparently, and supports efficient and real-time deployments. However, because LLM hallucination rates remain critically high and deterministic guardrails exhibit practical limitations, the semantic layer is explicitly positioned as a research prototype. It is designed to serve as an assistive cognitive tool that requires Human-in-the-Loop (HITL) oversight, and caution is advised against its use for automated incident response decisions in its current form.

An empirical evaluation of the benchmark datasets Edge-IIoTSet and IoTID20 showed that NEXUS-IDS achieves better detection accuracy and produces interpretable outputs that bridge technical attribution and human understanding. In addition to performance improvements, NEXUS-IDS offers a human-centered, transparent, and trustworthy intrusion detection framework for IoT networks.

This study offers the following key contributions.

1. **Novel Flow-Node Graph Neural Network Architecture:** We introduce a graph-based IDS in which network flows serve as nodes and communication patterns as edges, enabling effective relational learning via two-hop message passing while maintaining the flow-level classification granularity.
2. **Dual-Layer Explainability Framework with Hallucination Mitigation:** We propose a comprehensive explainability framework that combines (i) structural explainability via GraphLIME for feature-level attributions and (ii) semantic explainability via LLM-powered interpretation with entropy-aware uncertainty calibration.
3. **Entropy-Aware Semantic Generation with Uncertainty Quantification:** We develop a semantic interpretation methodology that conditions explanation generation on prediction uncertainty using Shannon entropy, dynamically injecting cautionary warnings for high-entropy predictions to communicate prediction uncertainty transparently, though hallucination prevention requires complementary mechanisms.
4. **Efficient LLM Deployment Architecture:** We designed a resource-efficient LLM integration strategy that uses entropy-based filtering and semantic caching, triggering LLM explanations for only $\sim 5\%$ of samples. We benchmarked the quality-latency trade-offs across four LLM architectures (82M–70B parameters).
5. **Rigorous LLM Quality and Human Expert Validation:** We employed Mixtral-8 $\times$ 22B as an independent judge to systematically evaluate semantic quality and factual accuracy. These automated metrics were explicitly validated through a structured human expert evaluation on a random sample of 96 explanations, confirming excellent inter-annotator agreement ($ICC = 0.91$) on explanation quality.
6. **Comprehensive Multi-Dataset Validation:** We validated the framework on the Edge-IIoTSet and IoTID20 datasets, reporting the detection accuracy, explanation fidelity, structural interpretability, semantic quality scores, hallucination rates, cache hit rates, and inference latency distributions.
7. **Trustworthy AI for Critical Infrastructure:** We established a human-centric design that prioritizes verifiable explanations, grounded attributions, and transparent communication of uncertainty, bridging high-performance detection and actionable security intelligence for industrial control systems and smart city infrastructure.

The remainder of this paper is organized as follows. [Section 2](#) reviews the related work on IoT intrusion detection, XAI, and LLMs in cybersecurity. [Section 3](#) details the proposed NEXUS-IDS framework and its design principles. [Section 4](#) outlines the core methodology, including graph construction, the IDS-FN-GNN model, structural explainability with GraphLIME, and the LLM-powered semantic explainability module. [Section 5](#) introduces the datasets and preprocessing procedures. [Section 6](#) presents the experimental evaluation and analysis, encompassing the experimental protocol, intrusion detection performance, structural interpretability, seman-

tic explainability, quality–latency trade-offs, hallucination assessment, and ablation studies. [Section 7](#) summarizes the key findings and discusses the main limitations. [Section 8](#) concludes the paper and outlines the future research directions.

2. Related work

The proposed NEXUS-IDS framework integrates three key research domains: IoT intrusion detection, Explainable AI (XAI), and the application of language models to cybersecurity. This section surveys the existing work in each domain, highlights major gaps, and clarifies how the proposed approach addresses these shortcomings.

2.1. IoT intrusion detection systems

IoT intrusion detection has evolved from traditional machine learning to sophisticated deep learning approaches. Ahmad et al. [9] applied supervised ML algorithms (Random Forest, SVM, ANN) to IoT-specific feature clusters from the UNSW-NB15 dataset, achieving 98.67% accuracy. However, their approach lacks explainability and operates as a black box, providing no insight into why specific traffic is flagged as being malicious. Hairab et al. [10] developed a CNN-based system for zero-day attack detection that incorporates attention visualization for basic interpretability. However, this single-layer mechanism provides only partial explainability, limited to attention-based visualization without systematic feature-level attribution, graph-aware structural explanation, or semantic interpretation. This restricts its utility for multi-stakeholder security communication in operational environments.

Graph neural networks have emerged as particularly promising tools for IoT security [11]. Lo et al. [12] proposed E-GraphSAGE, which captures both edge features and network topology, achieving strong detection performance. Nevertheless, the absence of an explainability mechanism renders the model's decisions unclear to security analysts. Although GNN-based IDS demonstrate high accuracy, they lack built-in graph-aware structural explanations and semantic interpretation features. Consequently, security analysts find it challenging to understand why certain traffic is flagged or how the model's graph-based reasoning leads to its detection decisions. These challenges inspired our flow-node GNN architecture, which incorporates dual-layer explainability.

2.2. Explainable AI in cybersecurity

The demand for transparency in AI-driven security systems has driven research into Explainable AI (XAI). Post hoc methods, such as LIME [13] and SHAP [14], identify important input features but operate on tabular data and do not naturally extend to graph-structured IoT network traffic data. For graph neural networks, Ying et al. [15] introduced GNNExplainer, which identifies critical subgraph structures, whereas Luo et al. [16] proposed PGExplainer for more efficient multi-instance explanations. Recent systematic reviews by Mohale and Obagbuwa [17] and Moustafa et al. [18] on XAI-integrated intrusion detection systems revealed that explainability features are critical for establishing analyst trust, enabling decision validation, and satisfying regulatory transparency requirements. Arreche et al. [19] proposed an end-to-end XAI framework for network intrusion detection that benchmarked seven black-box AI models across three real-world intrusion datasets. The framework employs the XAI methods LIME and SHAP to generate local and global explanations, addressing the opacity of AI decision-making processes and enhancing interpretability for security analysts.

Despite these advances, existing XAI-based IDS methods still lack graph-aware structural explanations and accessible, natural-language interpretations.

2.3. Large language models for security intelligence

LLMs have recently gained attention in cybersecurity because of their ability to generate human-readable explanations and support analyst decision-making. Yigit et al. [20] reviewed the roles of Generative AI, LLMs, and Agentic AI in critical infrastructure protection, highlighting opportunities in trust, privacy, resilience, and security. Houssel et al. [21] investigated the feasibility of using LLMs as Network Intrusion Detection Systems, comparing GPT-4 and LLaMA3 with traditional and transformer-based models for malicious NetFlow detection.

However, deploying LLMs for IoT intrusion detection remains challenging. Large models require substantial computational resources [22,23], and their outputs may contain hallucinations [24–26], which is particularly risky in security-critical environments. Moreover, existing LLM-based security systems often generate explanations that do not explicitly reflect the uncertainty of the model. NEXUS-IDS addresses these issues through entropy-aware prompting, semantic caching, and systematic hallucination assessments.

To evaluate explanation quality at scale, recent studies have introduced LLM-as-a-Judge frameworks [27,28], wherein a separate LLM assesses semantic quality and factual reliability. This approach aligns with human judgment [29] and provides a scalable alternative to manual evaluation.

2.4. Hybrid approaches for intrusion detection

Recent studies have combined graph-based learning, XAI, and language models for intrusion detection. Venkateswarlu et al. [30] combined GNNs with SHAP and LIME on UNSW-NB15 and CICIDS2017 datasets. Govea et al. [31] integrated GNNs with CyberBERT for semantic encoding, achieving strong detection performance and consistent explanations. Farrukh et al. [32] proposed XG-NID, which uses an LLM to generate explanations from integrated-gradient attributions.

Table 1
Comparison of IoT intrusion detection approaches.

Work	Method	XAI	LLM	Graph	Halluc-Ctrl	Unc-Quant	IoT-Data
Ahmad et al. [9]	ML	✗	✗	✗	✗	✗	✗
Hairab et al. [10]	CNN	✓ [*]	✗	✗	✗	✗	✓ [*]
Lo et al. [12]	GNN	✗	✗	✓	✗	✗	✓
Arreche et al. [19]	XAI-IDS	✓	✗	✗	✗	✗	✗
Houssel et al. [21]	LLM-NIDS	✓ [*]	✓	✗	✗	✗	✗
Venkateswarlu et al. [30]	GNN-XAI	✓	✗	✓	✗	✗	✗
Govea et al. [31]	GNN-BERT	✓	✓ [*]	✓	✗	✗	✓
Farrukh et al. [32]	HGNN + LLM	✓ [*]	✓	✓	✗	✗	✗
NEXUS-IDS	FN-GNN + GraphLIME + LLM	✓✓	✓	✓	✓	✓	✓✓

✓: Feature present; ✗: Absent; ✓^{*}: Partial (for XAI, indicates limited single-method explainability (e.g., attention visualization) without graph-aware structural explanation or semantic-level interpretation; for IoT data or LLM, indicates limited scope (e.g., single dataset or indirect use)); ✓✓: Advanced (multiple heterogeneous IoT datasets with comprehensive explainability evaluation).

Halluc-Ctrl: Explicit hallucination mitigation mechanisms (e.g., entropy calibration, grounded attribution, and systematic detection).

Unc-Quant: Uncertainty quantification integrated into explanation generation.

IoT-Data: Use of IoT-specific datasets for validation purposes.

Although these studies demonstrate the potential of hybrid IDS frameworks, they remain limited in three ways. First, SHAP/LIME-based explanations are not graph-aware and may be weakly grounded in the relational traffic structure. Second, LLM-generated explanations are often not explicitly anchored to verifiable structural attributions, increasing the risk of hallucination. Third, many evaluations rely on traditional intrusion datasets instead of heterogeneous IoT-specific environments.

2.5. Research gaps and positioning

Table 1 compares NEXUS-IDS with the most relevant studies. The reviewed literature reveals five key gaps: (i) limited use of dual-layer explainability that combines structural and semantic interpretation; (ii) insufficient uncertainty awareness in LLM-generated security explanations; (iii) lack of systematic hallucination assessment; (iv) limited validation on heterogeneous IoT-specific datasets; and (v) absence of an integrated framework that jointly supports graph-based detection, grounded explainability, and operational semantic interpretation.

NEXUS-IDS addresses these gaps by unifying graph-based intrusion detection, GraphLIME-derived structural explanations, and entropy-aware LLM-generated semantic narratives within a single framework. This integration improves the transparency, reliability, and operational interpretability of IoT intrusion detection in security-critical environments.

3. Proposed framework: NEXUS-IDS

NEXUS-IDS (Natural EXplanations from strUctural to Semantic for IoT Intrusion Detection Systems) is a human-centric, explainable intrusion detection framework designed for complex IoT environments. The framework integrates graph-based deep learning with a dual-layer explainability mechanism that combines structural explanations and semantic reasoning. Its primary objective is to deliver accurate and interpretable intrusion detection, thereby enabling security analysts to understand, trust, and act on model decisions.

Unlike conventional black-box intrusion detection systems, NEXUS-IDS distinctly separates detection, structural explanation, and semantic interpretation into integrated layers. This architecture combines high-performance detection with transparent feature-level insights and human-understandable threat descriptions.

3.1. Overall architecture

The NEXUS-IDS framework is organized into three core layers, as illustrated in Fig. 1: (1) a graph-based intrusion detection layer, (2) a dual-layer explainability layer, and (3) a Security Operations Center (SOC) layer.

3.1.1. Graph-based intrusion detection layer

At the core of NEXUS-IDS lies a Flow-Node Graph Neural Network (IDS-FN-GNN). Network traffic is transformed into a graph representation, where each network flow is modeled as a node, and communication endpoints (IP addresses) serve as contextual device nodes. The edges encode the relationships between the devices and flows, thereby enabling relational learning across traffic patterns.

The IDS-FN-GNN leverages graph convolutional layers to propagate information between related flows via shared communication endpoints. This flow-as-node design enables effective relational learning through two-hop message passing, allowing flows that share common endpoints to exchange contextual information indirectly while maintaining flow-level classification granularity. This architecture allows the model to capture both local flow characteristics and global interaction patterns, enabling the robust detection of diverse IoT attacks, including coordinated attacks such as botnets, scanning campaigns, and distributed denial-of-service patterns.

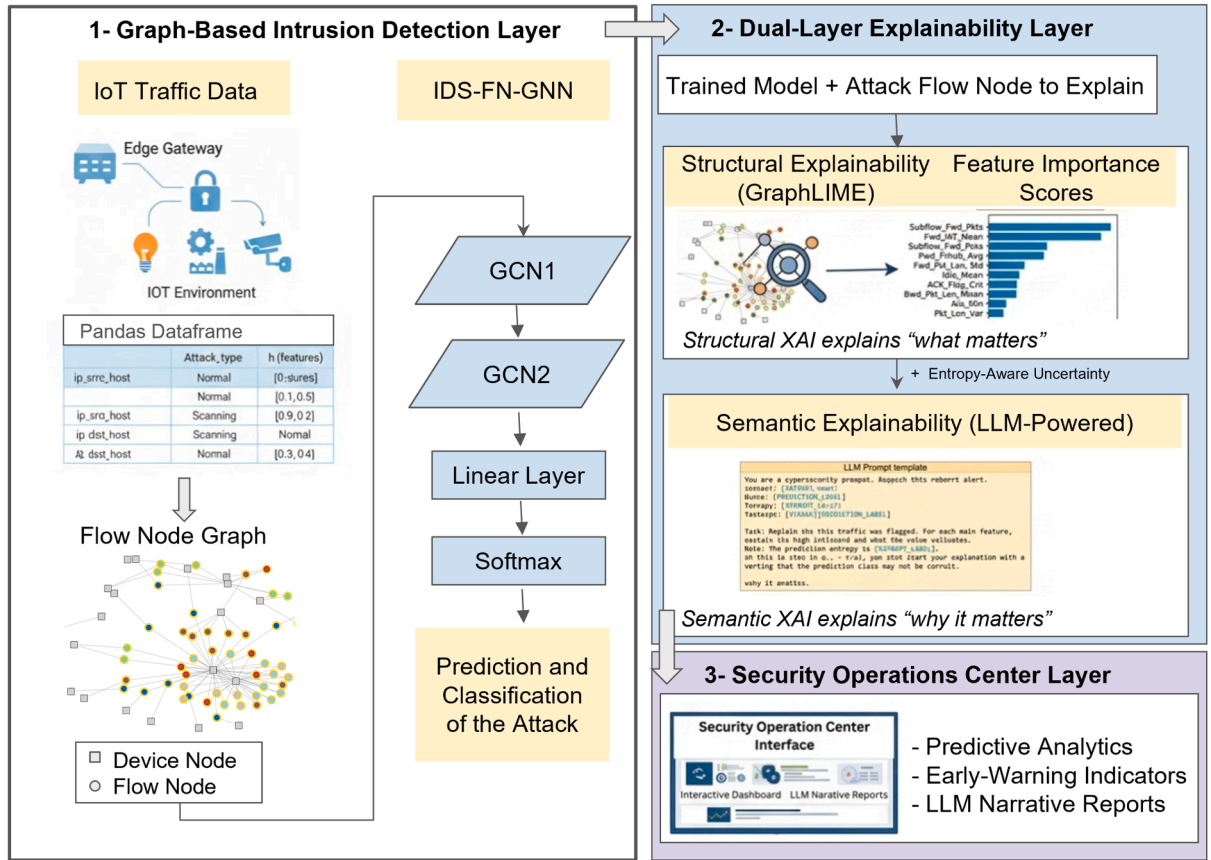


Fig. 1. Overview of the proposed NEXUS-IDS framework for IoT intrusion detection with dual-layer explainability. The first layer performs graph-based attack detection by transforming IoT traffic flows into a flow-device graph and processing them with the IDS-FN-GNN model. The second layer provides dual explainability: structural explanations using GraphLIME to identify the most influential features and graph neighborhoods, and semantic explanations generated by an LLM to produce human-readable security interpretations. The final outputs are integrated into a Security Operations Center (SOC) interface to support predictive analytics, early-warning indicators, and actionable security reports.

3.1.2. Dual-explainability layer

To enhance transparency and operational trust, NEXUS-IDS incorporates a dual-layer explainability mechanism that bridges the gap between technical model outputs and human understanding.

Structural Explainability via GraphLIME. GraphLIME was used as a post hoc explainability module to interpret the IDS-FN-GNN predictions. For a given flow node, GraphLIME fits a sparse, locally interpretable surrogate model over the node's neighborhood, producing feature-importance scores that quantify the contribution of individual traffic attributes to the predicted attack class. This layer provides precise feature-level explanations grounded in observable network behavior and graph topology.

Semantic Explainability via Large Language Models. Although GraphLIME produces quantitative feature attributions, these outputs remain technical and numerical, and are accessible primarily to data scientists and ML experts. To bridge the gap between model internals and human understanding, NEXUS-IDS integrates a semantic interpretation layer powered by LLMs. This layer transforms GraphLIME outputs, prediction confidence (entropy), and contextual metadata into coherent and analyst-readable threat narratives for the user. Given the current factual reliability limits of LLMs, this semantic module operates as an experimental research prototype rather than a fully autonomous operational utility.

The semantic layer employs three critical mechanisms to ensure reliability and efficiency.

- **Contextual Feature Enrichment:** GraphLIME importance scores are enriched with original normalized feature values from network flow data, grounding explanations in concrete observations rather than abstract attributions. This enables forensic statements such as "the ICMP sequence number exhibits unusually high variance (normalized value 0.92), consistent with flooding behavior."
- **Entropy-Aware Uncertainty Calibration:** Prediction uncertainty is quantified using Shannon entropy over the softmax probability distribution. When the entropy exceeds a configurable threshold, the system dynamically injects mandatory cautionary warnings

into the LLM prompts, requiring the model to preface reports with explicit acknowledgments of uncertainty. This mechanism mitigates overconfident explanations and reduces the risk of misleading semantic output.

- **Semantic Caching:** To enable real-time deployment while minimizing LLM inference costs, the system incorporates an entropy-based filtering and semantic caching architecture. Predictions with low entropy are logged immediately, whereas high-uncertainty samples are queried against a cache that stores dense feature vectors. Cache hits enable instant explanation retrieval, whereas cache misses trigger LLM inference and subsequent cache updates.

The framework supports a flexible spectrum of LLM architectures, ranging from lightweight models (e.g., DistilGPT2, ~82M parameters) deployable at edge gateways to foundation models (e.g., Llama-3.1-70B) hosted on centralized SOC infrastructure. This architectural flexibility enables organizations to balance the explanation quality, response latency, and computational cost according to their operational constraints.

3.1.3. Security operations center (SOC) layer

The final layer presents the detection results to security analysts through an interactive Security Operations Center (SOC) interface. Alerts are accompanied by structural explanations from GraphLIME (feature importance scores and induced subgraphs) and semantic narratives generated by the LLM. This dual presentation enables analysts to:

- Quickly assess threat severity and attack type through natural-language narratives
- Validate model decisions by examining technical feature attributions and graph structures
- Understand uncertainty levels through explicit entropy-calibrated warnings
- Make informed, evidence-based response decisions grounded in verifiable explanations

By integrating flow-based graph learning with a dual-layer explanation mechanism, NEXUS-IDS shifts intrusion detection from a purely predictive activity to a clear, interpretable, and reliable decision-support system that is well-suited for safeguarding critical infrastructure.

3.2. Key design principles

The NEXUS-IDS is built on three fundamental design principles:

1. **Grounded Explainability:** All semantic explanations are anchored to verifiable GraphLIME attributions, ensuring that natural language narratives remain faithful to actual model reasoning rather than fabricating plausible-sounding but incorrect interpretations.
2. **Uncertainty Transparency:** Prediction confidence is explicitly communicated through entropy quantification and calibrated prompting, ensuring analysts are aware of model limitations and ambiguous cases.
3. **Multi-Stakeholder Accessibility:** The dual-layer design provides technical attributions for ML experts and data scientists (GraphLIME) alongside human-readable narratives for SOC analysts and management (LLM), enabling effective communication across organizational roles.

4. Methodology: core components

This section presents the technical foundations of NEXUS-IDS, detailing the core methodological components that enable accurate detection and a transparent explanation. We first describe the IDS-FN-GNN architecture, including data preprocessing, graph construction strategies, and a two-hop message-passing mechanism that captures the relational patterns in IoT network traffic. We then present a dual-layer explainability framework comprising GraphLIME for structural feature-level interpretations and an LLM-powered semantic layer that translates technical attributions into human-readable threat narratives.

4.1. Core intrusion detection model

This subsection presents the proposed graph-based intrusion detection model, including the flow-as-node graph construction strategy and the IDS-FN-GNN architecture, which leverages two-hop message passing to capture relational patterns in IoT network traffic.

4.1.1. Flow-as-node graph construction

To enable flow-level classification and interpretability, we constructed a graph where network flows are modeled as nodes. The graph consists of two types of nodes.

- **Device nodes:** Each unique source or destination IP address is represented as a device node, modeling network entities such as IoT devices, gateways, and servers.
- **Flow nodes:** Each network flow record is represented as a flow node. These nodes are the main targets for the attack classification.

Formally, the node set is defined as follows: $\mathcal{V} = \mathcal{V}_d \cup \mathcal{V}_f$, where $\mathcal{V}_d$ and $\mathcal{V}_f$ denote the sets of device and flow nodes, respectively.

Traditional graph-based IDS aggregate flows into edge features [12], thereby collapsing temporal patterns and protocol anomalies that prevent the detection of subtle multi-step attacks. Our flow-as-node design preserves fine-grained flow characteristics while

enabling relational reasoning via two-hop message passing, thereby maintaining the per-flow granularity essential for coordinated attack detection and precise threat localization.

The edges encode the communication relationships between the devices and flows. For each flow node $f \in \mathcal{V}_f$ with source device s and destination device d , we add directed edges: $(s \rightarrow f)$, $(f \rightarrow d)$.

To enhance message passing and stabilize the training, reverse edges are added, resulting in a bidirectional graph structure. This design allows flows that share a common source or destination device to indirectly exchange information via shared neighbors.

Node Features and Labels. Node features are defined as:

$$\mathbf{x}_v = \begin{cases} \mathbf{0} \in \mathbb{R}^F, & \text{if } v \in \mathcal{V}_d, \\ \mathbf{f}_v \in \mathbb{R}^F, & \text{if } v \in \mathcal{V}_f. \end{cases} \quad (1)$$

Only the flow nodes are assigned attack labels. Device nodes are assigned an ignore label and excluded from the loss computation. Training and testing splits were performed exclusively on flow nodes, preventing label leakage through the graph structure.

4.1.2. Flow-node graph neural network (IDS-FN-GNN)

Model Architecture. Given the constructed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, we employ a two-layer Graph Convolutional Network (GCN) to perform flow-node classification. The shallow architecture was intentionally chosen to mitigate the over-smoothing and hub-dominance effects commonly observed in deeper GCNs on graphs with high-degree nodes.

Hub-Aware Device-Only Weighting. To address hub dominance caused by high-degree device nodes (e.g., gateways or broadcast addresses), we applied a device-only, hub-aware weighting strategy. Edges originating from device nodes are downscaled using a degree-based attenuation factor, whereas edges originating from flow nodes retain a unit weight.

Two-Hop Relational Message Passing. Due to the bipartite-like structure of the graph, relational learning occurs through a structured two-hop aggregation process:

- **Flow-to-Device (First Hop):** In the first GCN layer, each device node aggregates features from all incident flow nodes, capturing the collective traffic behavior as follows:

$$\mathbf{h}_v^{(1)} = \sigma \left(\sum_{f \in \mathcal{N}(v)} \frac{w_{vf}}{\sqrt{d_v d_f}} \mathbf{W}^{(0)} \mathbf{x}_f \right), \quad v \in \mathcal{V}_d. \quad (2)$$

Here, d_v and d_f denote the degrees of device node v and flow node f , respectively. Because messages in the first hop originate from flow nodes, the corresponding edge weights are set to $w_{vf} = 1$.

- **Device-to-Flow (Second Hop):** In the second layer, each flow node aggregates the contextual information from its associated device nodes as follows:

$$\mathbf{h}_f^{(2)} = \sigma \left(\sum_{v \in \mathcal{N}(f)} \frac{w_{fv}}{\sqrt{d_f d_v}} \mathbf{W}^{(1)} \mathbf{h}_v^{(1)} \right), \quad f \in \mathcal{V}_f. \quad (3)$$

Here, since messages in the second hop originate from device nodes, the hub-aware edge weight is defined as $w_{fv} = d(v)^{-\alpha}$, where $d(v)$ is the degree of the device node v and α controls the strength of the hub attenuation.

This mechanism enables flows that do not share a direct edge to influence each other through shared device nodes, facilitating the detection of coordinated attacks, such as botnets, scanning campaigns, and distributed denial-of-service attacks.

Classification and Training Objective. The final flow representation is passed through a linear classification head. To ensure compatibility with GraphLIME, the model outputs the softmax probabilities.

$$\hat{\mathbf{y}}_v = \text{softmax}(\mathbf{z}_v). \quad (4)$$

Training is performed using the cross-entropy loss computed exclusively over labeled flow nodes via a binary mask $\mathcal{M}$:

$$\mathcal{L} = - \sum_{v \in \mathcal{V}} \mathcal{M}_v \cdot \log P(y_v | \mathbf{h}_v), \quad (5)$$

where $\mathcal{M}_v = 1$ if $v \in \mathcal{V}_f$ and 0 otherwise. The device nodes participate only in message passing and do not contribute to gradient updates.

4.2. Multi-level explainability

To bridge the gap between technical model outputs and human understanding, the NEXUS-IDS implements a dual-layer explainability framework that operates at two complementary levels. The first layer uses GraphLIME to generate structural explanations through feature-level attributions grounded in graph topology and flow characteristics, providing precise technical insights for data scientists and security researchers. The second layer leverages LLMs to transform these numerical attributions into natural language semantic narratives accessible to security analysts and operational personnel. This subsection details the methodologies, architectures, and mechanisms underlying both explainability layers, including entropy-aware uncertainty calibration, contextual feature enrichment, and semantic caching strategies that ensure reliability and computational efficiency.

4.2.1. Structural explainability via GraphLIME

To improve the transparency and interpretability of our graph-based intrusion detection framework, we incorporated GraphLIME, a local post hoc explanation technique tailored for GNNs. Although GNNs are effective at modeling complex dependencies among network entities, their latent representations are often difficult to interpret, limiting their adoption in security-critical IoT environments. GraphLIME addresses this challenge by providing feature-level explanations of individual predictions.

In this work, we adopt the GraphLIME methodology proposed by Huang et al. [8] as the conceptual basis for structural explainability. Instead of relying on the original reference implementation, we developed a custom PyTorch Geometric-based adaptation to ensure compatibility with our flow-node graph representation and the IDS-FN-GNN architecture. Specifically, the explainer operates on individual flow nodes, extracts their local neighborhood subgraphs, and uses the trained IDS-FN-GNN outputs to fit a sparse surrogate model that identifies the most influential input features for every prediction. This preserves the original GraphLIME principles of local, post-hoc, and sparse feature-level explanation, while tailoring the implementation to the graph construction and node-classification pipeline used in NEXUS-IDS.

This method predicts a target node by fitting a sparse, locally interpretable surrogate model within the neighborhood of the node. For a given flow node $\mathcal{V}_f$, the explanation aims to identify the most influential input features that contribute to the predicted attack class.

$$\hat{y}_f = \arg \max_c P(y = c \mid \mathbf{x}_f, \mathcal{N}(\mathcal{V}_f)), \quad (6)$$

where $\mathbf{x}_f$ denotes the feature vector of flow node $\mathcal{V}_f$, and $\mathcal{N}(\mathcal{V}_f)$ represents the local neighborhood in the graph.

GraphLIME operates by locally approximating the trained IDS-FN-GNN with a sparse linear model that preserves the original graph structure. For each explained flow node, GraphLIME assigns an importance score to each input feature, quantifying its local contribution to the predicted attack class. These scores enable the direct interpretation of protocol-level and temporal traffic characteristics, such as ICMP sequence behavior, packet-size distributions, inter-arrival timing patterns, and transport-layer indicators.

At the flow level, GraphLIME provides a fine-grained insight into why a specific network flow is classified as benign or malicious. By exploring the corresponding feature importance scores, we can verify whether the predictions are driven by semantically meaningful traffic attributes rather than by structural artifacts, such as hub connectivity.

4.2.2. Semantic explainability via large language models

To complement the GraphLIME-based structural explanations, NEXUS-IDS incorporates an LLM-powered semantic explainability layer that translates numerical attributions into coherent, context-aware security narratives suitable for operational use in security operation centers. Although IDS-FN-GNN achieves high detection accuracy and GraphLIME provides precise feature-level explanations, these outputs remain technical and may not be directly actionable by human analysts.

It is important to emphasize that the semantic explainability module is strictly post hoc and does not participate in the intrusion classification. The IDS-FN-GNN prediction pipeline and the GraphLIME-based structural explanation mechanism remain fully functional without the LLM component. Therefore, the role of the LLM is limited to converting grounded technical attributions into human-readable narratives that support analyst interpretation and decision making.

Conditional Semantic Generation. The generation of semantic explanations is formulated as a conditional sequence modeling problem. Let $S = \{w_1, w_2, \dots, w_n\}$ denotes the sequence of tokens that comprise a natural language explanation. The LLM aims to generate an optimal explanation sequence $\hat{S}$ by maximizing the conditional likelihood:

$$\hat{S} = \arg \max_S \prod_{i=1}^n P(w_i \mid w_{<i}, \mathcal{T}(D, \hat{y}, \Phi, H)), \quad (7)$$

where $\mathcal{T}(\cdot)$ denotes a deterministic prompt construction function, D represents the dataset and deployment context, $\hat{y}$ is the predicted attack label, Φ is the GraphLIME feature importance vector for the target flow, and H denotes prediction uncertainty.

Entropy-Aware Uncertainty Conditioning. To ensure calibrated and trustworthy explanations, semantic generation is explicitly conditioned on predictive uncertainty, which is estimated using Shannon entropy over the softmax probability distribution.

$$H(p) = - \sum_{i=1}^C p_i \log p_i, \quad (8)$$

where p_i denotes the predicted probability of class i and C is the number of attack categories. Entropy serves as a confidence proxy; low values indicate decisive predictions, whereas high values reflect ambiguity among competing classes.

Throughout this paper, we adopt the following convention consistent with the activation rule $H > \tau$: a *permissive* threshold refers to a *lower* value of τ (more flows exceed the gate and are routed to the semantic layer), whereas a *strict* threshold refers to a *higher* value of τ (only the most uncertain predictions are forwarded).

As illustrated in Fig. 2, entropy governs the decision gate within the semantic module. When $H \leq \tau$, where τ is a predefined confidence threshold, the predictions are considered reliable, and the explanations are logged directly. When $H > \tau$, the prompt template dynamically injects a mandatory cautionary instruction, explicitly warning the analyst that the prediction may be uncertain. This entropy-aware prompting mechanism mitigates overconfident explanations and reduces the risk of misleading semantic output.

To analyze the sensitivity of this entropy-aware conditioning, we evaluated multiple threshold values $\tau \in \{0.1, 0.3, 0.5\}$. The entropy threshold controls the triggering of the semantic explanation module.

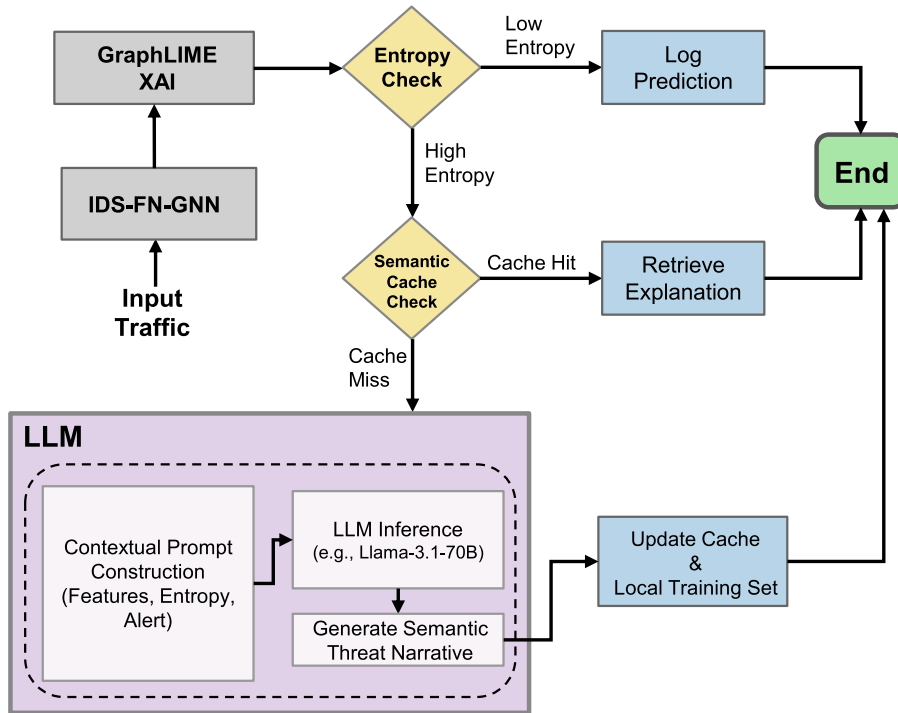


Fig. 2. Workflow of the LLM-powered semantic explainability module. Technical outputs are enriched with raw data context and calibrated by prediction entropy before being synthesized into narratives by the LLM.

Because the gate is defined as $H > \tau$, lower thresholds are *permissive* while higher thresholds are *strict*, directly governing the trade-off between explanation coverage and computational efficiency. At a permissive threshold ($\tau = 0.1$), between 27.13% and 39.92% of the total incoming flows exceed the gate and are routed to the semantic layer, increasing explanation availability while also increasing cache lookup and prompt-construction loads. Conversely, a strict threshold ($\tau = 0.5$) restricts forwarding to only the most uncertain predictions (14.67% to 22.90% of total flows), reducing coverage but tightening the alignment between semantic generation and genuine model ambiguity. Across all tested thresholds, the post-cache fresh LLM invocation rate remained remarkably stable at approximately 5% of the total flows, as the semantic cache effectively absorbed the volume of recurring structural patterns. The intermediate value $\tau = 0.3$ forwards 19.33% (Edge-IIoTSet) and 32.08% (IoTID20) of incoming flows, optimally balancing explanation coverage with computational selectivity, and is therefore adopted as the default setting throughout our experiments.

Structured Prompting and Semantic Translation. The structured prompt template shown in Figure A.1 (Appendix A) serves as a standardized and controlled linguistic interface that is applied identically across all four candidate LLM architectures. By using a uniform prompt structure across models, from the lightweight DistilGPT2 to the Meta Llama-3.1-70B foundation model, the framework ensures rigorous, fair evaluation of each model's innate semantic reasoning and translation capabilities. This template embeds the predicted class, entropy value, and most influential GraphLIME features augmented with their normalized raw values and protocol-level descriptions into a consistent and concise forensic instruction. By constraining the generation space through these fixed roles, tasks, and uncertainty cues, the LLM serves as a semantic translation layer that maps sparse numerical attributions into dense, human-readable, threat narratives.

Contextual Feature Enrichment. To ground explanations in observable network behavior, GraphLIME importance scores are enriched with the original normalized feature values extracted from flow records. Rather than reporting abstract importance alone (e.g., “feature X is influential”), the semantic layer enables statements such as “the ICMP sequence number exhibits unusually high variance (normalized value 0.92), consistent with flooding behavior.” This enrichment ensures that the explanations are forensic, verifiable, and actionable.

Operational Efficiency via Semantic Caching. To support real-time deployment and reduce the LLM inference overhead, NEXUS-IDS incorporates an entropy-aware semantic caching mechanism. High-confidence predictions are logged directly, whereas high-uncertainty samples are first queried against a semantic cache that stores previously generated explanations indexed by dense feature representations. A cache hit occurs when the cosine similarity between the incoming feature vector x and a stored vector y satisfies the following

equation:

$$\text{sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \geq \rho, \quad (9)$$

where ρ is the high-fidelity similarity threshold. We set $\rho = 0.98$ to balance cache efficiency with explanation fidelity: higher thresholds ($\rho > 0.99$) reduce cache hit rates by requiring near-identical feature vectors, whereas lower thresholds ($\rho < 0.95$) risk retrieving explanations for semantically distinct flows, which could mislead analysts. This conservative threshold ensures that cached explanations remain factually grounded in the flow characteristics while achieving practical cache utilization. In a cache hit, the stored explanation is retrieved instantly; otherwise, the sample is forwarded to the LLM for novel narrative generation, and the resulting explanation is added to the cache.

Through this entropy-aware, structured, and cache-supported semantic explainability layer, NEXUS-IDS elevates intrusion detection from a purely predictive task to a transparent, analyst-centric decision-support system. The integration of LLMs enables the framework to convey not only *what* type of attack was detected but also *why* the decision was made and *how confident* the system is in its assessment.

Algorithmic Workflow of the Semantic Module. Algorithm 1 details the execution logic of the semantic explanation and caching workflow. The routine begins with an *entropy gate* (Lines 1–4): predictions with low uncertainty ($H \leq \tau$) are logged directly and bypass the semantic layer entirely, ensuring that LLM resources are reserved for genuinely ambiguous cases. Only high-uncertainty samples ($H > \tau$) proceed to the *Fast Path* (Lines 5–8), which retrieves previously generated explanations for recurring traffic patterns based on vector similarity, enabling near-instantaneous responses without invoking the LLM. When no suitable semantic match is found, the system activates the *Slow Path* (Lines 13–14), triggering controlled LLM inference. Because the entropy gate has already filtered out confident predictions, every sample that reaches prompt construction (Lines 9–12) satisfies $H > \tau$ and therefore receives the mandatory uncertainty warning I_{warn} , ensuring that all LLM-generated narratives explicitly communicate the residual model uncertainty. Finally, the caching decision (Lines 15–16) ensures that newly generated explanations are vectorized and stored, progressively converting future queries from the Slow Path to the Fast Path.

Algorithm 1 Semantic explanation & caching routine.

Require: Feature Importance Φ , Raw Features $\mathbf{x}$, Prediction $\hat{y}$, Entropy H , Threshold τ , Similarity Threshold ρ

Ensure: Threat Narrative E (or logged prediction)

```

1: Step 1: Entropy Gate
2: if  $H \leq \tau$  then                                     ▷ Low uncertainty: bypass semantic layer
3:   return LOGPREDICTION( $\hat{y}$ ,  $\mathbf{x}$ )
4: end if
5: Step 2: Cache Lookup                                   ▷ High uncertainty only
6: if SIM( $\mathbf{x}$ , CACHE)  $> \rho$  then
7:   return RETRIEVE(CACHE,  $\mathbf{x}$ )                             ▷ Fast Path
8: end if
9: Step 3: Prompt Construction
10:  $D_{\text{context}} \leftarrow \text{ENRICH}(\Phi, \mathbf{x})$                      ▷ Map importance to raw values
11:  $I_{\text{warn}} \leftarrow \text{"Warning: High Uncertainty Detected..."}$    ▷ Always injected on this path, since  $H > \tau$ 
12:  $P \leftarrow \text{TEMPLATE}(D_{\text{context}}, \hat{y}, H, I_{\text{warn}})$ 
13: Step 4: LLM Inference
14:  $E \leftarrow \text{LLM}_{\text{generate}}(P)$                              ▷ Slow Path
15: Step 5: Caching Decision
16: UPDATECACHE( $\mathbf{x}$ ,  $E$ )                                     ▷ Store for future hits
17: return  $E$ 

```

LLM Architecture Considerations. To accommodate diverse deployment scenarios ranging from resource-constrained edge gateways to centralized SOC, NEXUS-IDS supports a flexible spectrum of LLM architectures. Lightweight models (e.g., DistilGPT2 with approximately 82M parameters) can be deployed at the edge to enable low-latency semantic explanations within strict computational budgets, whereas larger foundation models (e.g., Llama-3.1-70B) can be hosted on centralized SOC infrastructure to generate richer, more expressive forensic narratives. This architectural flexibility allows organizations to balance the explanation quality, response latency, and computational cost according to their operational constraints.

Summary of Semantic Explainability Capabilities. The LLM-powered semantic module in NEXUS-IDS provides the following key capabilities.

- **Grounded Semantic Explainability:** The numerical feature attributions produced by GraphLIME are translated into human-understandable explanations based on raw and normalized traffic feature values.
- **Hallucination-Aware Narratives with Explicit Uncertainty Communication:** Entropy-aware conditioning explicitly communicates prediction uncertainty, preventing overconfident or misleading explanations when the model confidence is low.

- **Efficient Real-Time Operation:** Entropy filtering and semantic caching significantly reduce the frequency of LLM invocations while maintaining the explanation fidelity.
- **Consistent Threat Interpretation:** Recurrent attack patterns yield stable semantic explanations, improving analyst trust and reducing cognitive load.

The synergy between GraphLIME and the LLM enables multi-layered explainability, spanning interpretable local feature contributions and high-level analyst-centric reasoning. This design bridges the gap between technical model outputs and actionable cybersecurity intelligence.

5. Datasets and preprocessing

To comprehensively validate the robustness, generalizability, and practical applicability of the proposed framework, we conducted experiments on two widely adopted and complementary IoT intrusion detection datasets. These datasets represent distinct IoT deployment scenarios, ranging from industrial edge environments to smart home IoT networks.

5.1. Edge-IIoTSet dataset

To evaluate NEXUS-IDS in realistic industrial edge environments, we used the Edge-IIoTSet dataset [33], which was designed for multi-class intrusion detection in IIoT settings. The dataset contains 15 classes: one normal class and 14 attack classes, covering DDoS, reconnaissance, injection-based attacks, malware, and MITM behaviors. Each record represents a network flow described by statistical, temporal, and protocol-level attributes.

Feature Selection and Cleaning. During preprocessing, irrelevant, sparse, text-heavy, and non-informative fields were removed, including raw payloads, HTTP URIs, MQTT messages, timestamps, duplicate labels, and low-level protocol metadata. After cleaning, 46 numerical flow-level features were retained as the node attributes. Attack labels were used for supervised learning, whereas the source and destination IP addresses were retained only as structural identifiers for graph construction.

IP Address Handling. Malformed or placeholder IP addresses, particularly in the MITM samples (e.g., 0 or 0.0.0.0), were handled using sentinel identifiers and explicit attacker nodes to preserve the graph connectivity and MITM-specific communication structure.

5.2. IoTID20 dataset

In addition to Edge-IIoTSet, we evaluated NEXUS-IDS on the IoTID20 dataset [34], which represents a smart home IoT environment with multiple IoT devices, user services, and external servers. The dataset contains CICFlowMeter-based flow records with 86 original features describing the packet counts, byte statistics, flow rates, inter-arrival times, and TCP flag behavior. We adopted a multi-class setting with five categories: *Normal*, *Mirai*, *Scan*, *DoS*, and *MITM ARP Spoofing*.

Feature Selection and Cleaning. Identifiers and non-behavioral fields, including Flow_ID, Timestamp, Label, and Sub_Cat, were removed. After preprocessing, 79 standardized numerical flow-level features were retained as the node attributes. The source and destination IP addresses were excluded from the feature vectors and used only as structural identifiers for constructing device nodes, whereas the attack labels were retained for supervised learning.

5.3. Unified data preprocessing and feature engineering

A unified preprocessing pipeline was applied to all datasets to ensure consistency and fair comparison. Duplicate records and samples with missing values were removed, and numerical features were standardized using z-score normalization: $x' = \frac{x - \mu}{\sigma}$, where μ and σ were computed only from the training split to prevent test set leakage. Categorical attributes, excluding the IP addresses, were label-encoded with a dedicated token for the unseen categories during inference. This strategy preserves the discriminative flow-level behavior while modeling device relationships through the graph structure, enabling the IDS-FN-GNN to learn robust representations across heterogeneous IoT environments.

6. Experimental evaluation and analysis

This section presents a comprehensive empirical evaluation of the NEXUS-IDS framework. Section 6.1 describes the experimental protocol and implementation. Section 6.2 reports the intrusion detection performance on the Edge-IIoTSet and IoTID20 datasets, including baseline comparisons and hub-aware weighting ablation. Section 6.3 analyzes the structural explainability using GraphLIME and compares it with alternative XAI methods. Section 6.4 evaluates the LLM-powered semantic explainability module, including model selection, operational efficiency, latency profiling, quality assessment, and hallucination detection.

Table 2

Overall classification performance on the IoTID20 and Edge-IIoTSet datasets. Results are reported as mean $\pm$ standard deviation over five independent runs with different random seeds.

Dataset	Accuracy (%)	Macro-F1 (%)	Weighted-F1 (%)
IoTID20	97.42 $\pm$ 0.32	97.41 $\pm$ 0.34	97.42 $\pm$ 0.31
Edge-IIoTSet	94.30 $\pm$ 0.48	93.22 $\pm$ 0.52	94.32 $\pm$ 0.46

6.1. Experimental protocol

All experiments related to IDS-FN-GNN training, graph construction, and GraphLIME-based structural explainability were implemented using PyTorch 1.13 and PyTorch Geometric 2.5. These experiments were conducted on a workstation equipped with an NVIDIA RTX A5000 GPU and 64 GB of system memory. The LLM-based semantic explainability experiments were performed on a server with four NVIDIA A100-SXM4 80 GB GPUs to support inference and evaluation across multiple LLM architectures.

The IDS-FN-GNN detector was implemented as a lightweight two-layer GCN operating on the flow-device graph. Each GCN layer used 64 hidden units with ReLU activation, and a dropout of 0.02 was applied between layers. To reduce hub dominance from high-degree device nodes, we used a device-only, hub-aware edge-weighting strategy. Edges originating from device nodes were scaled by $d(v)^{-\alpha}$, where $d(v)$ is the degree of device node v and $\alpha = 0.5$, while edges originating from flow nodes retained a unit weight.

Each dataset was randomly shuffled and partitioned into training, validation, and test sets using a stratified 70%/15%/15% split, ensuring that the original class distribution was preserved across all subsets. No explicit class-balancing strategy, such as oversampling, undersampling, class-weighted loss, or focal loss, was applied during IDS-FN-GNN training. Therefore, to avoid relying only on accuracy under imbalanced conditions, we reported the macro-F1, weighted-F1, and per-class precision, recall, F1-score, and support values. Five independent runs with fixed random seeds were conducted to estimate the performance variance. The model was optimized using Adam with a learning rate of 1×10^{-3} and a weight decay of 5×10^{-4} for 150 epochs.

We adopted a transductive graph learning setting in which all flow nodes were included in the constructed graph to preserve the complete communication topology. During training, only the labels of the training flow nodes were used for loss computation, whereas the validation and test labels were masked. Therefore, label leakage did not occur.

For structural explainability, GraphLIME was applied as a local post hoc explainer on the flow device graph. For each target flow node, a two-hop neighborhood ($k = 2$) was extracted, and a sparse surrogate model was fitted to approximate the local decision-making behavior of the IDS-FN-GNN. A Gaussian RBF kernel with width $\rho = 0.01$ was used, and ℓ_1 regularization was applied to promote sparsity. The final explanation retained the top- $K = 10$ features, ranked by absolute importance.

6.2. Intrusion detection performance

The detection performance of IDS-FN-GNN was assessed on the Edge-IIoTSet and IoTID20 datasets, which cover heterogeneous IoT traffic, diverse attack categories, and imbalanced class distributions. The analysis reports the overall classification results, per-class behavior, baseline comparisons, and ablation findings to examine the model's robustness against both high-volume and stealthy attacks.

6.2.1. Dataset 1: Edge-IIoTSet

As reported in Table 2, the proposed model achieved an overall accuracy of 94.30%, with a macro-averaged F1-score of 93.22% and a weighted F1-score of 94.32%. These results demonstrate the model's strong ability to generalize across both majority and minority attack classes. In addition, the high macro-averaged scores indicate that the performance is consistently strong across different attack categories, rather than being dominated by frequent classes such as Normal or DDoS traffic.

Per-Class Analysis. The model demonstrated near-perfect detection performance for volumetric and protocol-based attacks, including DDoS ICMP, DDoS TCP, and DDoS UDP, all of which achieved F1-scores above 99%, as illustrated in Table 3. This indicates that the proposed graph-based representation effectively captures the protocol-specific temporal and structural patterns associated with large-scale flooding attacks.

Strong performance is also observed for Normal traffic, with a precision of 98.87% and recall of 98.83%, confirming the model's ability to distinguish benign behavior even when normal flows coexist with multiple types of attack.

Analysis of Stealthy Attack Performance. Although IDS-FN-GNN achieves near-perfect performance on volumetric attacks ($F1 > 99\%$ for DDoS_TCP, DDoS_UDP, and DDoS_ICMP), its performance is lower for stealthy application-layer attacks, including Password (84.68%), Fingerprinting (83.72%), XSS (88.12%), Uploading (87.36%), and SQL_Injection (87.29%). This gap is mainly attributed to sparse local graph neighborhoods, class imbalance, and overlapping traffic patterns among stealthy attacks.

Volumetric attacks typically generate dense clusters around shared source-destination pairs, allowing two-hop message passing to reinforce the malicious signal. In contrast, stealthy attacks often appear as isolated or low-degree flows embedded within mostly benign sessions, thereby limiting the benefits of relational aggregation. Their under-representation in the training set further reduces the model's ability to learn stable class-specific graph patterns.

Table 3

Per-class classification performance on the Edge-IIoTSet dataset (Mean $\pm$ Std over five runs). Support denotes the total test-set count.

Class	Support	Precision (%)	Recall (%)	F1-score (%)
Backdoor	2039	99.58 $\pm$ 0.21	93.62 $\pm$ 0.68	96.51 $\pm$ 0.37
DDoS_HTTP	2112	98.02 $\pm$ 1.95	84.47 $\pm$ 1.74	90.74 $\pm$ 1.41
DDoS_ICMP	2818	100.00 $\pm$ 0.00	99.96 $\pm$ 0.04	99.98 $\pm$ 0.02
DDoS_TCP	2050	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
DDoS_UDP	2900	99.83 $\pm$ 0.09	100.00 $\pm$ 0.00	99.91 $\pm$ 0.05
Fingerprinting	200	100.00 $\pm$ 0.00	72.00 $\pm$ 2.83	83.72 $\pm$ 1.91
MITM	162	100.00 $\pm$ 0.00	96.91 $\pm$ 0.62	98.43 $\pm$ 0.32
Normal	4860	98.87 $\pm$ 0.34	98.83 $\pm$ 0.47	98.85 $\pm$ 0.41
Password	1998	87.97 $\pm$ 3.02	81.63 $\pm$ 1.28	84.68 $\pm$ 1.96
Port_Scanning	2014	93.54 $\pm$ 2.20	96.28 $\pm$ 1.84	94.89 $\pm$ 1.65
Ransomware	2185	91.37 $\pm$ 2.62	95.01 $\pm$ 2.03	93.16 $\pm$ 1.68
SQL_Injection	2062	79.29 $\pm$ 3.58	97.09 $\pm$ 2.12	87.29 $\pm$ 2.17
Uploading	2054	85.78 $\pm$ 2.96	89.00 $\pm$ 3.90	87.36 $\pm$ 2.44
Vulnerability_Scanner	2015	98.14 $\pm$ 0.41	91.46 $\pm$ 2.31	94.68 $\pm$ 1.64
XSS	2011	86.68 $\pm$ 2.05	89.61 $\pm$ 1.63	88.12 $\pm$ 1.72

Table 4

Per-class classification performance on the IoTID20 dataset (Mean $\pm$ Std over five runs). Support denotes the total test-set count.

Class	Support	Precision (%)	Recall (%)	F1-score (%)
DoS	8,909	100.00 $\pm$ 0.00	99.00 $\pm$ 0.41	99.50 $\pm$ 0.20
MITM ARP Spoofing	5,307	92.96 $\pm$ 1.21	99.22 $\pm$ 0.35	95.99 $\pm$ 0.74
Mirai	62,352	97.63 $\pm$ 0.84	93.43 $\pm$ 1.12	95.48 $\pm$ 0.96
Normal	6,011	98.27 $\pm$ 0.43	99.50 $\pm$ 0.22	98.88 $\pm$ 0.31
Scan	11,290	98.45 $\pm$ 0.52	95.98 $\pm$ 0.78	97.20 $\pm$ 0.61

As shown in Fig. 3, most errors occur among stealthy attack classes rather than between malicious and Normal traffic. Password, SQL_Injection, and Uploading flows are not misclassified as Normal, and only 3 XSS flows (0.15%) leak into the Normal class. This indicates that the model generally preserves the malicious-versus-benign boundary, whereas confusion mainly arises among attacks with similar HTTP-level behaviors. In such cases, the explainability layer can help analysts refine the final attack interpretation.

6.2.2. Dataset 2: IoTID20

As illustrated in Table 2, the IDS-FN-GNN achieved a strong overall performance on the IoTID20 dataset, with an accuracy of 97.42%, a macro-F1 score of 97.41%, and a weighted-F1 score of 97.42%. These results indicate a balanced detection performance across the five traffic categories.

Per-Class Analysis. Table 4 reports the per-class results for IoTID20. The model achieved near-perfect performance for DoS, with 100.00% precision, 99.00% recall, and 99.50% F1-score, reflecting its ability to identify high-volume attack patterns. Normal traffic was also accurately identified, achieving an F1-score of 98.88%, confirming the model's ability to distinguish benign IoT behavior from malicious activity.

The model maintained a strong performance for more challenging attack types. MITM ARP Spoofing achieved 99.22% recall and 95.99% F1-score, while Mirai reached an F1-score of 95.48%, indicating reliable detection of stealthy man-in-the-middle and botnet-related behaviors. Scan traffic also achieved a high F1-score of 97.20%, indicating that the model can detect reconnaissance patterns that may partially overlap with benign traffic. The consistently high recall and F1-scores across all IoTID20 classes suggest that IDS-FN-GNN provides robust and balanced intrusion detection in smart home IoT environments.

6.2.3. Comparison with additional baselines

To further validate the effectiveness of the proposed IDS-FN-GNN architecture, we extended the experimental evaluation by including several additional baselines covering both traditional machine learning models and graph neural network architectures.

Specifically, we implemented Random Forest (RF), GraphSAGE, and Graph Attention Network (GAT). All models were trained using the same feature set, preprocessing pipeline, and dataset splits as the IDS-FN-GNN to ensure a fair comparison. In particular, we used the same stratified data partition, feature normalization procedure, and fixed random seed. For the graph-based baselines, GraphSAGE and GAT were trained on the same device-flow bipartite graph representation using comparable hyperparameters (hidden dimension = 64, 2 layers, dropout = 0.02, learning rate = 0.001, 150 epochs).

The results in Table 5 demonstrate the effectiveness of the proposed IDS-FN-GNN compared with traditional and graph-based baselines. On Edge-IIoTSet, IDS-FN-GNN achieved 94.30% accuracy and 93.22% macro-F1, outperforming GraphSAGE by 5.15 pp in accuracy and 6.29 pp in macro-F1. It also exceeded GAT by 16.67 pp in accuracy and 24.20 pp in macro-F1. The RF baseline achieved an accuracy of 68.2323%, confirming that tabular feature aggregation is less effective than graph-based relational modeling for IoT traffic.

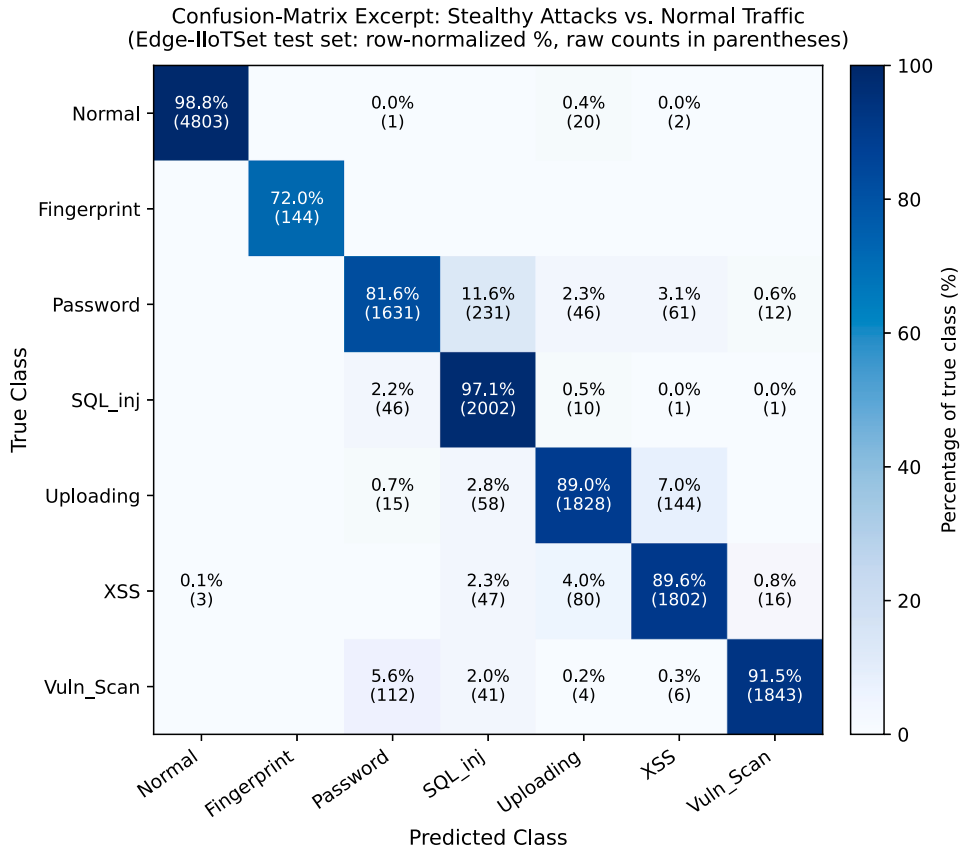


Fig. 3. Confusion-matrix excerpt for stealthy attack classes and Normal traffic on the Edge-IIoTSet test set.

Table 5

Comparison with additional baselines on Edge-IIoTSet and IoTID20 datasets.

Method	Edge-IIoTSet		IoTID20	
	Acc.	Macro-F1	Acc.	Macro-F1
Random Forest (RF)	68.23	56.98	72.15	60.41
GraphSAGE	89.15	86.93	91.78	89.64
GAT (2-head)	77.63	69.02	80.45	72.11
IDS-FN-GNN (ours)	94.30	93.22	97.42	97.41

A similar trend was observed in IoTID20, where IDS-FN-GNN achieved the best performance, with 97.42% accuracy and 97.41% macro-F1. Compared with GraphSAGE, the strongest baseline, IDS-FN-GNN improved the accuracy by 5.64 pp and macro-F1 by 7.77 pp. It also substantially outperformed GAT and RF, which achieved 80.45% and 72.15% accuracy, respectively.

These results highlight the importance of modeling device-flow relationships through graph-based representations. By combining graph-aware aggregation with hub-aware weighting, IDS-FN-GNN consistently outperformed both the traditional RF baseline and alternative GNN architectures across the two datasets, demonstrating its ability to capture the relational dependencies in heterogeneous IoT traffic.

While this subsection benchmarks IDS-FN-GNN against alternative detection architectures (RF, GraphSAGE, and GAT), [Section 6.3.3](#) provides a complementary comparison of GraphLIME with alternative explainability methods, including Gradient Saliency, Integrated Gradients, and GNNExplainer.

6.2.4. Ablation study: hub-aware weighting

To assess the contribution of the proposed hub-aware weighting mechanism, we compared four message aggregation variants: (i) no weighting, (ii) uniform dampening with a constant factor of 0.5, (iii) global hub weighting applied to all nodes, and (iv) device-only hub-aware weighting with $\alpha = 0.5$, which represents the proposed configuration. All variants were evaluated using the same preprocessing pipeline, graph construction procedure, and training settings for both the Edge-IIoTSet and IoTID20 datasets.

Table 6

Ablation study of the hub-aware weighting mechanism on Edge-IIoTSet and IoTID20 datasets.

Variant	Edge-IIoTSet		IoTID20	
	Acc.	Macro-F1	Acc.	Macro-F1
No weighting (baseline)	92.39	91.05	95.41	95.12
Uniform dampening (0.5)	91.56	90.57	94.87	94.66
Global hub weighting	94.15	93.23	97.21	97.10
Device-only hub ($\alpha = 0.5$)	94.30	93.22	97.42	97.41

Table 6 shows that hub-aware weighting improves the performance over the unweighted and uniformly dampened variants. On Edge-IIoTSet, removing the weighting mechanism reduced the accuracy from 94.30% to 92.39%, whereas uniform damping further reduced it to 91.56%. On IoTID20, the same trend was observed, with the unweighted baseline achieving 95.41% accuracy compared to 97.42% for the proposed device-only weighting strategy.

These results indicate that naive down-weighting of all nodes is ineffective for handling structural imbalances in the device-flow graphs. In contrast, selectively attenuating high-degree device nodes reduces hub dominance while preserving the meaningful flow relationships. Collectively, the ablation confirms that device-only hub-aware weighting contributes to a more robust graph representation learning across both datasets.

6.2.5. IDS-FN-GNN inference latency

To characterize the computational cost of the detection component, we measured per-flow inference latency on the trained IDS-FN-GNN and the real Edge-IIoTSet graph. In batch mode (full-graph forward pass), the model completed the inference in 5.37 ± 0.39 ms, yielding a per-flow latency of 0.034 ± 0.002 μ s, four orders of magnitude below any practical network alert threshold. In the online (single-sample) mode, where one flow's 2-hop subgraph is forwarded independently, the per-query latency is 3.86 ± 0.26 ms (p50 = 3.69 ms, p95 = 4.38 ms, p99 = 4.48 ms). These results confirm that the IDS-FN-GNN detection component operates in genuine real-time.

6.3. Structural explainability with GraphLIME

To enhance the transparency, interpretability, and trustworthiness of the proposed framework, we employ GraphLIME to provide instance-level explanations by approximating the decision boundary of the model in the local neighborhood of a target node. In the context of IoT intrusion detection, such explainability is essential not only for validating the correctness of the model but also for ensuring that the predictions are grounded in meaningful traffic semantics rather than spurious structural artifacts.

6.3.1. Feature importance for Edge-IIoTSet

Fig. 4 presents a representative GraphLIME explanation for a correctly classified Edge-IIoTSet flow in the test dataset. The left panel shows the induced local flow device subgraph, and the right panel shows the most influential features driving the prediction.

This shows that IDS-FN-GNN relies mainly on protocol- and transport-layer attributes, including UDP stream characteristics, checksum-related features, and connection-state indicators. These features are consistent with DDoS traffic behavior, suggesting that the prediction is driven by meaningful network patterns rather than by spurious artifacts. The local subgraph further illustrates how the shared device nodes provide a relational context that supports the classification decision. These GraphLIME feature-importance scores were then passed to the semantic LLM module to generate human-readable explanations.

6.3.2. Feature importance for IoTID20

Fig. 5 presents a representative GraphLIME explanation for a correctly classified IoTID20 flow. The left panel shows the induced local flow device subgraph, and the right panel shows the most influential features driving the prediction.

This indicates that IDS-FN-GNN relies mainly on temporal and transport layer attributes, including SYN flag counts, flow duration, and ACK flag statistics. These features are consistent with the scanning behavior, where systematic probing produces distinctive connection and timing patterns. The induced subgraph further shows how the shared device nodes provide a relational context that supports the classification decision. As with Edge-IIoTSet, the resulting GraphLIME feature-importance scores were forwarded to the semantic LLM module to generate human-readable explanations.

6.3.3. Benchmarking GraphLIME against explainability baselines

To rigorously justify the selection of GraphLIME as the structural explanation component in NEXUS-IDS, we benchmarked it against three established and widely used attribution methods: Gradient Saliency, Integrated Gradients [35], and GNNExplainer [15]. All methods were applied to the same trained IDS-FN-GNN model, and the same set of evaluation flows from the Edge-IIoTSet dataset.

Evaluation metrics. We use four complementary metrics: Spearman rank correlation, Jaccard feature overlap, explanation stability, and Fidelity@10.

Spearman rank correlation (ρ) measures the agreement between the ranked feature importance lists produced by two explanation methods. A value close to 1 indicates strong agreement in feature ranking, while a value close to 0 indicates weak or no correlation.

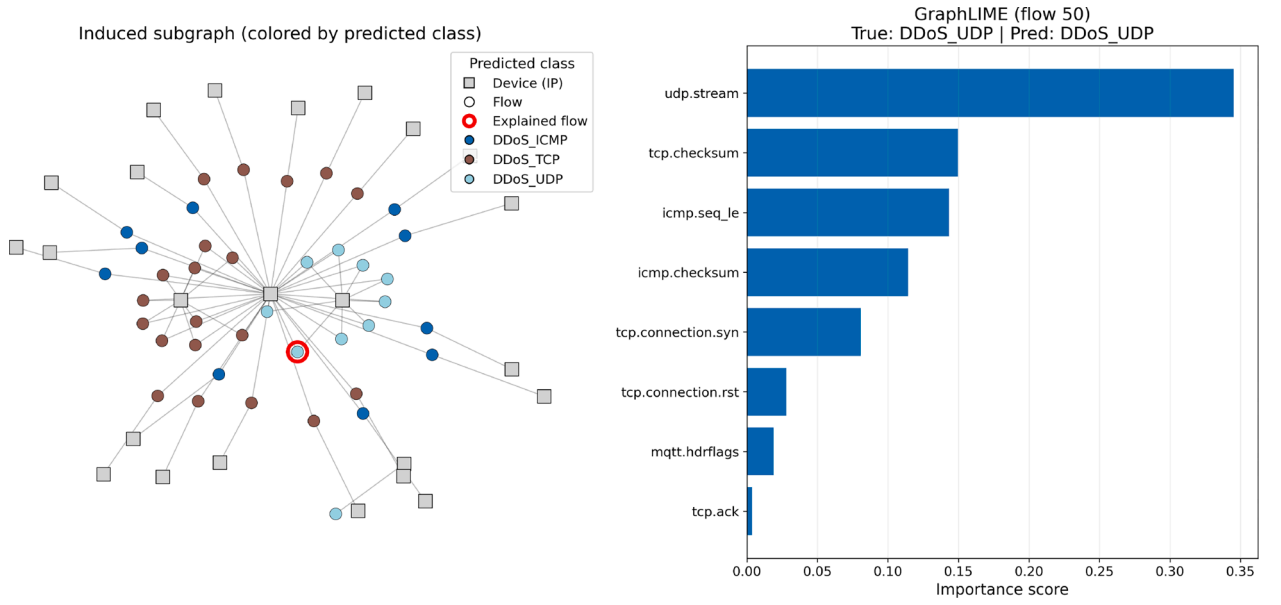


Fig. 4. Two-panel GraphLIME explanations for correctly classified flows in the Edge-IIoTSet dataset. The induced subgraph (left) illustrates the local flow–device context, while feature importance scores (right) highlight attributes driving the prediction.

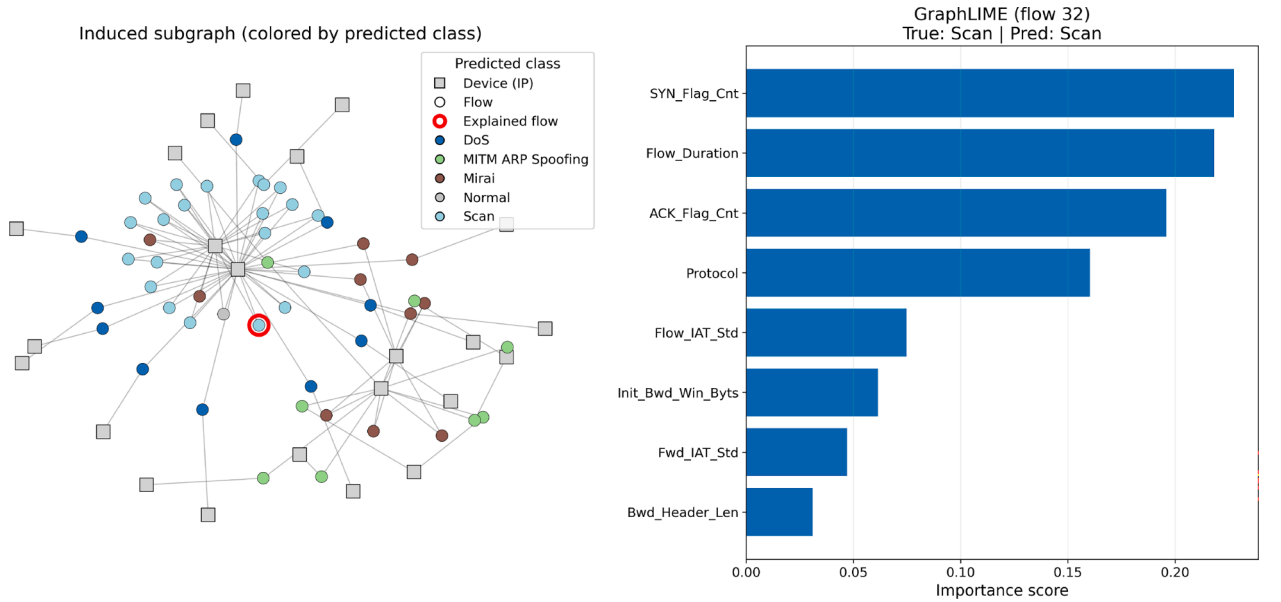


Fig. 5. Two-panel GraphLIME explanations for correctly classified flows in the IoTID20 dataset.

Jaccard similarity measures the overlap between the top- k most important features identified by two methods. For a given k , it is defined as the ratio between the intersection and union of the two feature sets: $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$, where A and B denote the top- k feature sets from the two explanation methods. Higher values indicate a stronger agreement.

Explanation stability measures the consistency of feature-importance scores across flows predicted to belong to the same attack class. Let $\mathcal{F}_c = \{f_1, f_2, \dots, f_{N_c}\}$ denote the set of flows predicted as class c , and let $\phi_{f_i} \in \mathbb{R}^F$ be the feature importance vector produced for flow f_i . The stability for class c is defined as the mean per-feature standard deviation:

$$\text{Stability}(c) = \frac{1}{F} \sum_{j=1}^F \sigma_j(c), \quad \sigma_j(c) = \sqrt{\frac{1}{N_c - 1} \sum_{i=1}^{N_c} (\phi_{f_i, j} - \bar{\phi}_{\cdot, j}(c))^2} \quad (10)$$

Table 7

Comparison of GraphLIME with alternative XAI approaches using explanation agreement, stability, and the faithfulness metric Fidelity@10, evaluated on the Edge-IIoTSet dataset.

Method	Spearman	Jaccard@5	Jaccard@10	Jaccard@15	Stability (95% CI)	Fidelity@10
Gradient Saliency	0.4108	0.428 ± 0.274	0.292 ± 0.113	0.424 ± 0.153	0.1146, [1.108, 1.183] × 10 ⁻¹	–
Integrated Gradients	0.2091	0.196 ± 0.080	0.398 ± 0.114	0.475 ± 0.161	0.0915, [8.84, 9.46] × 10 ⁻²	–
GNNExplainer	0.4457	0.144 ± 0.095	0.316 ± 0.124	0.494 ± 0.178	0.0522, [4.98, 5.47] × 10 ⁻²	–
GraphLIME (ours)	–	–	–	–	1.45 × 10⁻⁴, [1.38, 1.53] × 10⁻⁴	0.934 ± 0.018

where $\bar{\phi}_{\cdot,j}(c)$ denotes the mean importance of feature j within class c . The reported stability score is the macro-average across the attack classes, with lower values indicating more consistent explanations. This variance-based metric captures the stability of the attribution magnitude and remains comparable across dense and sparse explainers. We report 95% confidence intervals using non-parametric bootstrap resampling with 1000 resamples.

Fidelity@10 measures the faithfulness of GraphLIME explanations to the IDS-FN-GNN decision process. For each explained flow node, the ten most important features identified by GraphLIME were retained, whereas all remaining features were masked. The modified sample was then passed through the trained IDS-FN-GNN again. Fidelity@10 is defined as the proportion of samples for which the predicted class remains unchanged after the feature retention process. Higher Fidelity@10 values indicate that the attributed features preserve the information most relevant to the model decision.

Results. Table 7 summarizes the comparison between GraphLIME and the alternative explainers. GraphLIME shows moderate agreement with the baselines, with Spearman correlations ranging from $\rho = 0.21$ to $\rho = 0.45$ and Jaccard@10 overlap ranging from 0.29 to 0.40. The overlap further increases at larger k , reaching up to 49% for Jaccard@15 with GNNExplainer. More importantly, GraphLIME achieves substantially lower stability scores than the alternative methods, with non-overlapping 95% confidence intervals. In addition, GraphLIME achieved a high Fidelity@10 score of 0.934 ± 0.018 , indicating that the top-ranked features preserved the IDS-FN-GNN prediction in most evaluated cases. This suggests that the GraphLIME attributions are not only faithful to the model decision process but also stable across the same-class flow. Therefore, the results support the use of GraphLIME as the primary structural explainer in NEXUS-IDS and as a grounded basis for the semantic LLM layer.

6.3.4. GraphLIME attribution latency

To characterize the computational cost of the structural explainability component, we measured per-flow attribution latency on the same trained model and graph. Across 30 representative test flows with 2-hop neighborhood sizes ranging from 2 to 154 nodes (mean = 32), GraphLIME attribution completes in 9.48 ± 3.47 ms per explained flow (p50 = 8.10 ms, p95 = 16.97 ms). This latency is acceptable for two reasons. First, GraphLIME is never applied to every incoming flow: after entropy filtering ($\tau = 0.3$), only 19–32% of flows are forwarded to the semantic layer, and the post-cache fresh invocation rate is $\approx 5\%$ of the total flows. Second, attribution runs asynchronously after the primary GNN detection decision and does not introduce latency into the alerting path.

6.4. Semantic explainability with LLM

This subsection evaluates the LLM-powered semantic layer that translates GraphLIME’s technical attributions into human-readable threat narratives. We first justify the selection of four candidate LLM architectures (82M–70B parameters) and the Mixtral-8x22B judge model (Section 6.4.1) and then present a comprehensive evaluation across three dimensions: operational efficiency via entropy-based caching, inference latency profiling across model sizes, and rigorous quality assessment, including both semantic coherence scores and systematic hallucination detection (Section 6.4.2).

6.4.1. Experimental spectrum: LLM selection rationale

To evaluate semantic explainability across heterogeneous IoT deployment scenarios, we selected a spectrum of LLMs ranging from lightweight edge-oriented models to large, centralized models. The selected models vary in terms of parameter count, architecture, and reasoning capability, enabling the analysis of trade-offs between explanation quality, latency, and computational cost.

Four candidate LLMs were evaluated for generating security explanations, ranging from approximately 82M to 70B parameters. The objective was to identify models capable of producing useful and operationally meaningful explanations under different hardware constraints, from edge gateways to centralized security servers.

In addition, Mixtral-8x22B was employed as an independent evaluator within an *LLM-as-a-Judge* framework to assess the explanation quality and detect hallucinations. This evaluation strategy has recently emerged as an effective, scalable alternative to human assessment [27,28,36,37], with recent benchmarks demonstrating a strong alignment between advanced open-weight judge models and human evaluations [29].

The evaluated models are listed in Table A.1 (Appendix A). To support deployment feasibility, the Llama-3.1-70B model was served through Ollama (v0.13.0) using 4-bit Q4_0 quantization, reducing the VRAM requirement from approximately 140 GB to approximately 40 GB while preserving high-quality semantic reasoning. Streaming was disabled for all models to ensure consistent latency measurements.

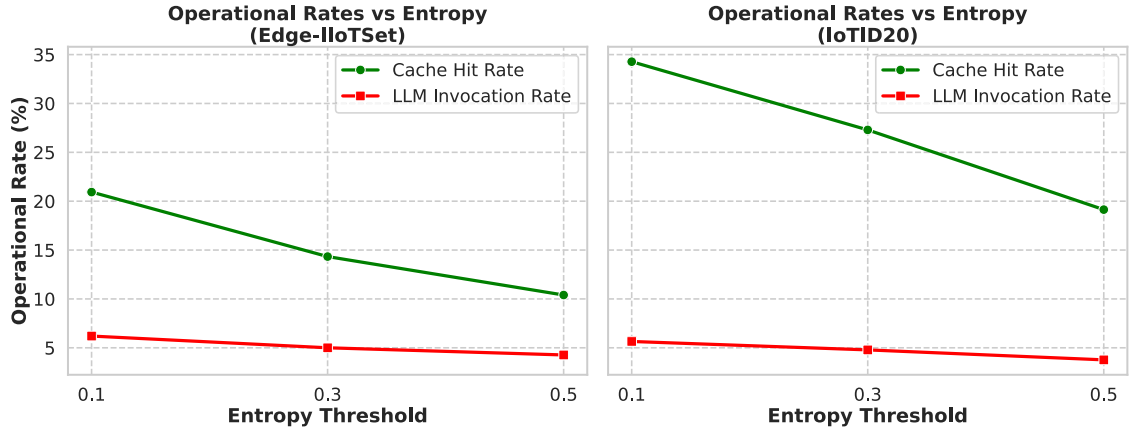


Fig. 6. Operational metrics analysis comparing cache hit rate and LLM invocation rate as the entropy threshold varies from permissive ($\tau = 0.1$) to strict ($\tau = 0.5$), for the Edge-IIoTSet and IoTID20 datasets.

The decoding settings are also reported for reproducibility. DistilGPT2, Flan-T5-Large, and Falcon-7B-Instruct used greedy decoding with a maximum generation length of 150 tokens. Flan-T5-Large also used beam search with four beams. Llama-3.1-70B used temperature = 0.7 and top-p = 0.9, whereas the Mixtral-8x22B judge model operated with greedy decoding in FP16 precision.

6.4.2. Results

The proposed architecture was evaluated on the Edge-IIoTSet and IoTID20 datasets across three dimensions: (1) Operational Efficiency, measuring the reduction of expensive model invocations via entropy-based caching; (2) Latency Profiling, quantifying end-to-end inference latency across model scales (DistilGPT2 to Llama-3.1-70B); and (3) Quality Assessment, employing Mixtral-8x22B as an LLM-as-a-Judge to evaluate semantic explanation quality and factual hallucination rates.

Operational Pipeline Breakdown. For clarity, we explicitly trace how an incoming flow propagates through the framework at the default threshold $\tau = 0.3$. Starting from 100% of incoming flows, the entropy filter forwarded only those with $H > \tau$ to the semantic layer; this corresponded to exactly 19.33% of flows on Edge-IIoTSet and 32.08% on IoTID20, computed as fractions of the total incoming flows. Within this forwarded subset, the semantic cache resolves the majority through similarity-based retrieval, yielding cache hit rates of 14.33% and 27.29% of the total incoming flows, respectively. The residual cache misses, which trigger fresh LLM inference, represent exactly 5.00% of the total flows for Edge-IIoTSet and 4.78% for IoTID20, relative to the total incoming flows. Throughout the remainder of the paper, the term *LLM invocation rate* consistently refers to this post-cache figure, computed relative to the total number of incoming flows.

Resource Utilization vs. Entropy Threshold. Fig. 6 demonstrates that our methodology bypasses computationally expensive LLM inference for the vast majority of network traffic, minimizing operational overhead compared with baseline approaches. At a permissive entropy threshold ($\tau = 0.1$), the system forwards a larger fraction of flows to the semantic layer, which the cache absorbs to yield high Cache Hit Rates ($\approx 21\%$ for Edge-IIoTSet and $\approx 34\%$ for IoTID20). As the threshold becomes stricter at $\tau = 0.5$, fewer flows are routed to the semantic layer and cache hits decrease ($\approx 10\text{--}19\%$), while the post-cache LLM invocation rate remains consistently low ($\approx 5\%$). Thus, the architecture effectively reserves heavy LLM inference strictly for complex, anomalous traffic samples.

Inference Latency Distribution. Fig. 7 reveals clear latency disparities across the model sizes. The latency distribution in Fig. 7 reports fresh LLM invocation latency only, i.e., cache-miss cases in which the model actually generates a new semantic explanation. These values exclude low-entropy bypassed flows and semantic cache hits. DistilGPT2 ($\approx 82\text{M}$ parameters) achieved near-real-time performance with a median latency of ≈ 0.6 s, whereas Llama-3.1-70B exceeded 10 s, reflecting its higher computational cost. This median fresh-invocation latency should not be directly compared with the end-to-end cached pipeline medians reported later in Tables 12 and 13, which include bypassed and cached cases. Intermediate models (Falcon-7B and Flan-T5-Large) fall within 2–5 s. These results motivate a hybrid deployment in which Llama-3.1 is reserved for high-entropy outliers, allowing smaller models and caching to handle the bulk of the traffic, while deep reasoning is used only where strictly necessary.

Deployment Metrics and Hardware Constraints. Table 8 summarizes the mean fresh-invocation latency, throughput (tokens/s), and peak VRAM across the models. Unlike the median latency in Fig. 7, Table 8 reports mean latency over fresh LLM invocations; therefore, differences between Fig. 7 and Table 8 reflect both different summary statistics and the same cache-miss-only measurement setting. DistilGPT2 has a negligible memory footprint (0.16 GB) and the highest throughput (25.54 tokens/s), making it edge-viable despite its limited reasoning quality. Its higher mean latency of 3.33 s, compared with the ≈ 0.6 s median in Fig. 7, indicates that a small number of slower fresh generations increase the average latency. Llama-3.1-70B requires 40.00 GB of VRAM even under 4-bit quantization,

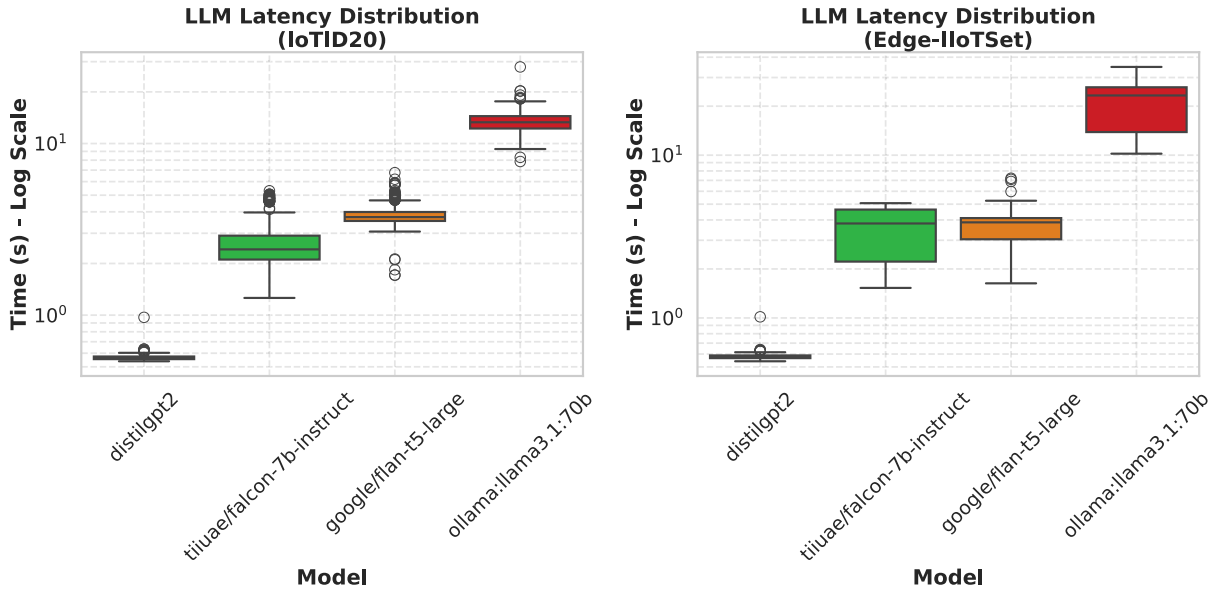


Fig. 7. LLM fresh-invocation inference latency distribution (log scale) for IoTID20 and Edge-IIoTSet datasets, comparing DistilGPT2, falcon-7b-instruct, flan-t5-large, and llama3.1:70b models. Only cache-miss generation cases are included; low-entropy bypasses and semantic cache hits are excluded.

Table 8

Deployment metrics: mean latency measured strictly during fresh LLM invocations, excluding semantic cache hits and low-entropy bypassed samples, throughput, and peak memory usage across candidate LLMs.

Model	Mean Fresh-Invocation Latency (s)	Throughput (Tokens/Sec)	Peak Memory Usage (GB)
DistilGPT2	3.33	25.54	0.16
google/flan-t5-large	17.53	0.68	0.98
tiuae/falcon-7b-instruct	14.21	4.00	6.89
ollama:llama3.1:70b	15.75	13.82	40.00

Note. The latency values in this table correspond to fresh LLM invocations only, that is, cache-miss cases in which a new semantic explanation is generated. They exclude low-entropy bypassed flows and semantic cache hits. Therefore, these values should be interpreted as model-generation latency rather than full-end-to-end semantic pipeline latency. The lower median latency reported for DistilGPT2 in Fig. 7 and the higher mean latency reported here are not contradictory, because they use different summary statistics over the fresh-invocation latency distribution.

Table 9

Precision-recall trade-offs for selected entropy thresholds (τ).

Threshold (τ)	Precision	Recall	F1-Score	False Alarms
0.1	0.406	0.882	0.556	2668
0.3	0.438	0.742	0.551	1964
0.5	0.465	0.579	0.516	1376

firmly positioning it as a centralized SOC asset. Flan-T5-Large exhibited the lowest throughput (0.68 tokens/s) despite moderate memory use (0.98 GB), a consequence of its encoder-decoder architecture on verbose network feature inputs.

Entropy Threshold Optimization. To justify $\tau = 0.3$, we evaluated prediction entropy as a misclassification classifier via Precision-Recall analysis (Fig. 8). The results summarized in Table 9 indicate that at a permissive threshold of $\tau = 0.1$, the system achieves the highest mathematical F1-score (0.56) driven by a high recall (0.88). However, a low precision (0.40) triggers 2668 false alarms, inducing severe analyst “warning fatigue” in operational SOCs. Furthermore, as previously noted in Fig. 6, a permissive $\tau = 0.1$ routes a higher volume of correct low-risk traffic into the semantic layer (inflating the Cache Hit Rate). Conversely, the stricter $\tau = 0.5$ reduces false alarms but drops recall to 0.57, allowing a dangerous false confidence in erroneous predictions. Thus, $\tau = 0.3$ serves as the optimal operational elbow (Table 9), maintaining a near-peak F1-score (0.55) while minimizing LLM invocations ($\approx 5\%$) and false alarms. This balance is further justified by class-specific baseline entropy variations (Tables 10 and 11), as certain correctly classified attacks exhibit naturally elevated uncertainty.

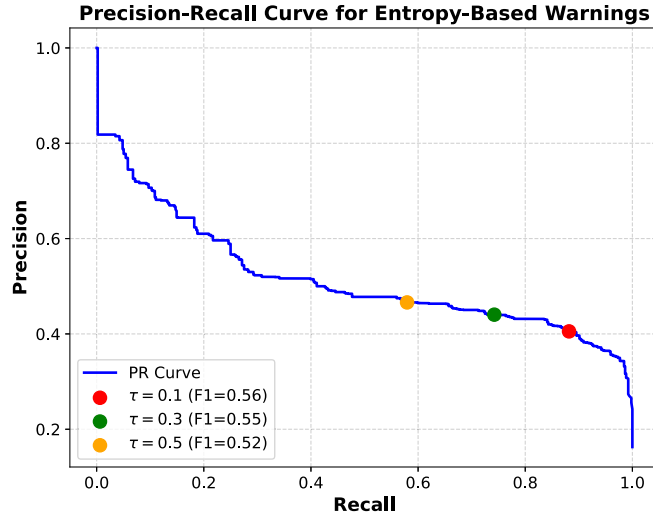


Fig. 8. Precision-recall trade-off for entropy-based uncertainty calibration. The curve evaluates the prediction entropy as a classifier for model misclassifications. Thresholds $\tau \in \{0.1, 0.3, 0.5\}$ illustrate the balance between detecting errors (Recall) and reducing false alarms (Precision).

Table 10

Complete class-specific entropy distributions for IoTID20.

Predicted Class	Count	Mean Entropy (Correct)	Mean Entropy (Error)
DoS	6492	0.457	0.485
MITM	5220	0.394	0.491
Mirai	6240	0.102	0.640
Scan	6540	0.040	0.486
Normal	6108	0.000	0.731

Table 11

Filtered class-specific entropy distributions for Edge-IIoTSet (excluding classes with mean error entropy < 0.2).

Predicted Class	Count	Mean Entropy (Correct)	Mean Entropy (Error)
DDoS_HTTP	1476	0.354	0.936
DDoS_UDP	1128	0.294	0.725
DDoS_TCP	1020	0.209	0.508
MITM	1236	0.162	0.876
DDoS_ICMP	1140	0.112	0.452
Vulnerability_scanner	1068	0.101	0.481
Fingerprinting	1320	0.099	0.561
Backdoor	1032	0.068	0.407
Ransomware	1200	0.067	0.608
Uploading	1344	0.058	0.416
Port_Scanning	1224	0.000	0.207

Per-Class Entropy Analysis and Adaptive Thresholding. Filtered class-specific distributions (Tables 10 and 11) justify the baseline threshold $\tau = 0.3$. First, the error state entropy systematically exceeded the correct state entropy across all classes where errors occurred (e.g., IoTID20 Mirai: 0.640 vs. 0.102; Edge-IIoTSet MITM: 0.876 vs. 0.162), confirming that predictive uncertainty is a highly reliable misclassification risk indicator. Second, $\tau = 0.3$ exceeds the mean correct state entropy for most classes. The exceptions involved complex signatures: DoS (0.457) and MITM (0.394) in IoTID20, and DDoS_HTTP (0.354) in Edge-IIoTSet. Conversely, four highly deterministic classes omitted from the Edge-IIoTSet visualization (Password, SQL_injection, Normal, and XSS) consistently maintained a mean correct-state entropy below 0.03. For the remaining classes, the static threshold successfully bypasses the LLM for confident predictions while routing ambiguous flows to the semantic translation layer.

To optimize throughput for classes with naturally elevated baselines (DoS, MITM, volumetric DDoS), future implementations can utilize class-adaptive thresholds:

$$\tau_c = \mu_c^{\text{correct}} + k \cdot \sigma_c^{\text{correct}} \quad (11)$$

where μ_c^{correct} and $\sigma_c^{\text{correct}}$ represent the per-class mean, and standard deviation of predictive entropy from correctly classified validation samples, and k is a global sensitivity coefficient. For perfectly identified classes exhibiting near-zero variance (specifically

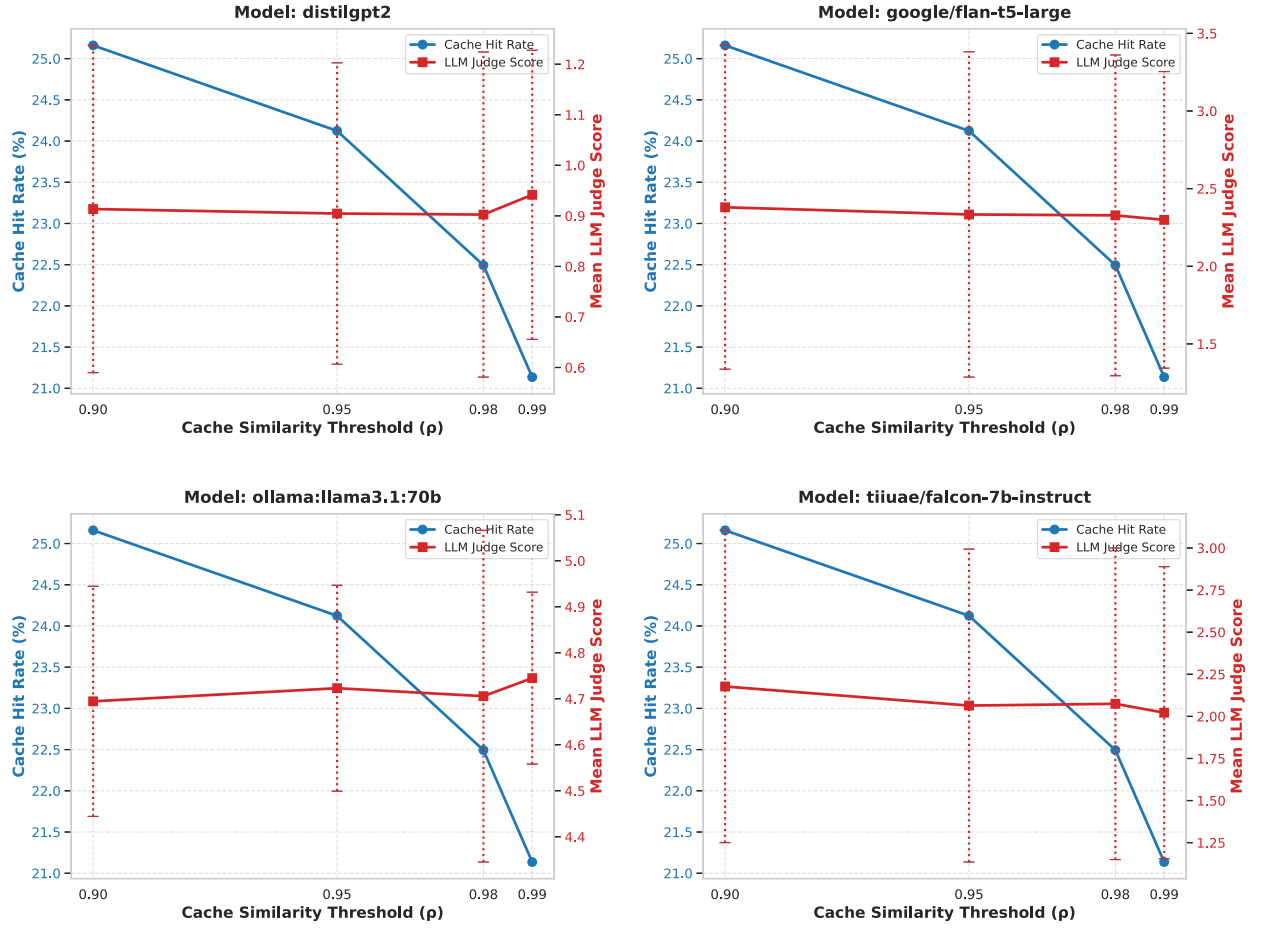


Fig. 9. Impact of the cache similarity threshold (ρ) on hit rates and explanation fidelity for each candidate model. The statistics were aggregated across both the IoTID20 and Edge-IIoTSet datasets. The dotted vertical bars indicate the standard deviation of the LLM Judge scores.

SQL_injection, Normal, and XSS in Edge-IIoTSet, where $\sigma_c^{\text{correct}} \approx 0$), this dynamic cutoff collapses tightly to $\tau_c \approx \mu_c^{\text{correct}}$, bypassing routine traffic while remaining highly sensitive to structural anomalies. This formulation preserves operational simplicity via a single tunable hyperparameter and lightweight $O(1)$ lookups, ensuring full architectural compatibility with the existing filters and caches. Empirically calibrating k across the profiles established in Tables 10 and 11 remains a focused avenue for future investigation.

Semantic Cache Threshold Sensitivity. To validate the semantic cache similarity threshold (ρ) in Eq. (9), we evaluated $\rho \in \{0.90, 0.95, 0.98, 0.99\}$. Fig. 9 shows the trade-off between Cache Hit Rate and Mean LLM Judge Score across Edge-IIoTSet and IoTID20. As expected, stricter thresholds reduced the cache hits for all models. Lower thresholds (e.g., $\rho = 0.90$) produced a “shielding effect” for lightweight and mid-tier models by reusing cached narratives and artificially boosting the quality scores. As ρ approached 0.99, most samples triggered cache misses, exposing the weaker reasoning capabilities of models such as DistilGPT2 and Falcon-7B-Instruct, and reducing their fidelity scores. In contrast, Llama-3.1-70B maintained consistently high scores across all thresholds, demonstrating a strong native reasoning ability. Overall, $\rho = 0.98$ provided the best balance between semantic fidelity and computational efficiency.

Table A.2 (Appendix A) further illustrates the shielding effect and the risks of overly permissive caching. For mid-tier models on Node 3438 (googleflan-t5-large and tiuae-falcon-7b-instruct), the 0.95 threshold incorrectly retrieved cached explanations from factually different flows (Feat_78, http.request.version). The stricter 0.98 threshold forced cache misses, generating grounded explanations based on the correct attributes (udp.stream, mbtcp.trans_id) and improving scores by 2.0–3.0 points. For llama3.1:70b, stricter thresholds enabled the refinement of formatting and narrative quality. DistilGPT2 was excluded because its outputs consisted largely of hallucinated repetition loops regardless of threshold, making cache misses operationally insignificant.

Performance Analysis and Trade-offs. To overcome the well-known limitations of n-gram metrics (BLEU, ROUGE) in evaluating semantic correctness and logical coherence [38,39], we utilized the state-of-the-art open-weight Mixtral-8x22B model as an independent evaluator within the LLM-as-a-Judge framework. Its Mixture-of-Experts (MoE) architecture provides robust reasoning comparable to proprietary APIs without closed-source data privacy risks. The evaluation prompt (Figure A.2, Appendix A) requires standardized

Table 12

Performance statistics (Mean, STD, Median, and IQR) and hallucination rate for the IoTID20 dataset. Reported time values reflect end-to-end pipeline execution averaged over both fresh LLM invocations and semantic cache hits.

Model	Time (s)				Score				Halluc.
	Mean	Std	Med.	IQR	Mean	Std	Med.	IQR	Rate (%)
DistilGPT2	0.0855	0.2021	0.0008	0.0002	0.7221	0.2685	0.5	0.5	99.83
falcon-7b-instruct	0.3996	1.0101	0.0008	0.0002	2.0860	0.9763	2.0	0.5	17.89
flan-t5-large	0.5771	1.3936	0.0008	0.0002	1.9131	0.4698	2.0	0.5	14.05
llama3.1:70b	2.0067	4.8422	0.0010	0.0008	4.3399	0.8280	4.8	0.8	34.26

Note. The reported median time corresponds to the end-to-end semantic pipeline latency over the evaluated operational flow stream. It includes low-entropy bypasses, semantic cache hits and fresh LLM invocations. Therefore, sub-millisecond medians mainly reflect the dominance of bypassed and cached cases and should not be directly compared with the fresh-invocation latency reported in Table 8.

numeric outputs for evaluating causal clarity and feature value integration and imposes a dedicated penalty for omitting high-entropy disclosures.

Formalization of the Hallucination Metric. We formally define the Hallucination Rate (HR) as the proportion of generated explanations containing factually incorrect, fabricated, or logically inconsistent security information. Using Mixtral-8x22B as an automated judge leverages its strong technical alignment with human experts to detect fabrications. The prompt (Figure A.3, Appendix A) enforces a strict binary scoring criterion (Yes/No) via a three-step verification: (1) mapping attack descriptions, (2) checking for fabricated features, ports, protocols, or CVEs, and (3) verifying logical consistency.

Table A.3 (Appendix A) enumerates qualitative successes and failures across models, detailing model/dataset contexts, node IDs, predictions, ground truth labels, explanations (with intrinsic bolding), and judge scores. Operationally, DistilGPT2 (score: 0.5–1.0) collapses into severe stochastic repetition loops (e.g., repeating “The probability of a given event”), correctly receiving near-zero utility scores. Falcon-7B-Instruct generates grammatically coherent prose (score: ~2.8–4.2) but applies generic templates, frequently justifying high feature influence by stating “it is a unique identifier” regardless of actual feature mechanics. Flan-T5-Large demonstrates inconsistent grounding; while successfully mapping numerical values for standard protocols (e.g., `http.content_length` of -0.31), it defaults to N/A placeholders on technical vectors like Node 4951. Its 4.8 score on the Edge-IIoTSet sample indicates potential *fluency bias*, where structural adherence is rewarded despite shallow forensic synthesis. Conversely, Llama-3.1-70B achieves justified near-perfect scores (4.8) by integrating prediction entropy metadata into structured warnings and synthesizing numerical feature meanings (e.g., linking a low `http.content_length` to server flood attempts) to deliver a tier-1 SOC analytical depth.

Quantitative comparisons (Tables 12 and 13, Fig. 10) reflect these dynamics. Llama-3.1-70B achieves peak quality (Mean Score: 4.34 for IoTID20, 4.27 for Edge-IIoTSet) but requires mean execution times of 2.01 s and 5.22 s, respectively. Conversely, DistilGPT2 exhibits negligible median end-to-end semantic pipeline latency (< 0.001 s), because most evaluated cases are resolved through low-entropy bypasses or semantic cache hits. However, this value should not be interpreted as fresh-generation latency: Table 8 reports only the cache-miss LLM invocation latency. Despite its lightweight size, DistilGPT2 remains unsuitable for semantic threat narration because its explanation-quality scores remain below 1.0 and its outputs are largely unusable. Including the Interquartile Range (IQR) clarifies these distributions; while Llama-3.1-70B excels in reasoning, its Edge-IIoTSet Time IQR (11.05 s) confirms significant latency variance when processing complex feature sets.

Pairwise Wilcoxon signed-rank tests indicate that all model-quality differences are highly statistically significant ($p < 0.001$, Table 14) over commonly evaluated paired flow samples (n). The paired design reduces the sample-level variability by comparing the model outputs for the same evaluated flows. Cohen’s d follows the direction of each comparison in Table 14: a positive value indicates that Model A scored higher than Model B, whereas a negative value indicates that Model A scored lower than Model B. Comparisons involving Llama-3.1-70B generally produced large to very large effects, particularly against DistilGPT2 and Falcon-7B-Instruct. The Flan-T5-Large vs. Falcon-7B-Instruct comparison showed a small negative effect on IoTID20 ($d = -0.23$), indicating that Falcon-7B-Instruct slightly outperformed Flan-T5-Large on this dataset. Conversely, the same comparison shows a medium positive effect on Edge-IIoTSet ($d = 0.75$), indicating that Flan-T5-Large outperformed Falcon-7B-Instruct on the higher dimensional Edge-IIoTSet benchmark. This dataset-dependent reversal suggests that the relative ranking of mid-sized models is sensitive to dataset characteristics and feature-space complexity.

A critical dimension of our evaluation is the hallucination rate, which is defined as the proportion of explanations containing fabricated or inaccurate security information. The results revealed major differences across the models, particularly for Edge-IIoTSet. DistilGPT2 reached 100

It is important to note that hallucination estimates from the LLM judge may themselves contain bias or reasoning errors. Accordingly, the reported hallucination rates should be interpreted as approximate comparative indicators rather than absolute ground truth. To mitigate hallucinations in future work, Retrieval-Augmented Generation (RAG) or external knowledge base verification [24] represent promising directions. Although we incorporated entropy-aware uncertainty calibration using Shannon entropy $H(\mathbf{p}) = -\sum p_i \log p_i$ to inject cautionary warnings when the uncertainty exceeded a threshold τ , this alone did not eliminate fabri-

Table 13

Performance statistics (Mean, Std, Median, and IQR) and hallucination rate for the Edge-IIoTSet dataset. Reported time values reflect end-to-end pipeline execution averaged over both fresh LLM invocations and semantic cache hits.

Model	Time (s)				Score				Halluc.
	Mean	Std	Med.	IQR	Mean	Std	Med.	IQR	Rate (%)
DistilGPT2	0.1473	0.2525	0.0007	0.5462	0.9668	0.2754	1.0	0.0	100.00
falcon-7b-instruct	0.8948	1.6499	0.0007	1.6159	1.7587	0.9384	1.5	1.0	96.51
flan-t5-large	0.9120	1.6217	0.0008	1.6835	2.5453	1.1361	2.5	1.3	53.11
llama3.1:70b	5.2236	9.5627	0.0012	11.0542	4.2695	1.0383	4.8	0.8	54.85

Note 1. The reported median time corresponds to the end-to-end semantic pipeline latency over the evaluated operational flow stream. It includes low-entropy bypasses, semantic cache hits, and fresh LLM invocations. Therefore, sub-millisecond medians mainly reflect the dominance of bypassed and cached cases and should not be directly compared with the fresh-invocation latency reported in [Table 8](#).

Note 2. The hallucination rate reported in this table corresponds to the end-to-end semantic pipeline evaluation for the corresponding dataset/model settings. It should be interpreted as a pipeline-level semantic reliability metric and not as the guardrail-confusion denominator reported later in [Table 15](#).

Table 14

Statistical significance and effect size analysis of LLM judge scores (Wilcoxon signed-rank test and Cohen's d).

Dataset	Model Comparison (A vs. B) [†]	p -value	Cohen's d	n
IoTID20	DistilGPT2 vs. flan-t5-large	< 0.001	−3.11	801
	DistilGPT2 vs. falcon-7b-instruct	< 0.001	−1.90	801
	DistilGPT2 vs. llama3.1:70b	< 0.001	−5.88	801
	flan-t5-large vs. falcon-7b-instruct	< 0.001	−0.23	801
	flan-t5-large vs. llama3.1:70b	< 0.001	−3.61	801
	falcon-7b-instruct vs. llama3.1:70b	< 0.001	−2.49	801
Edge-IIoTSet	DistilGPT2 vs. flan-t5-large	< 0.001	−1.91	394
	DistilGPT2 vs. falcon-7b-instruct	< 0.001	−1.15	394
	DistilGPT2 vs. llama3.1:70b	< 0.001	−4.35	394
	flan-t5-large vs. falcon-7b-instruct	< 0.001	0.75	394
	flan-t5-large vs. llama3.1:70b	< 0.001	−1.58	394
	falcon-7b-instruct vs. llama3.1:70b	< 0.001	−2.54	394

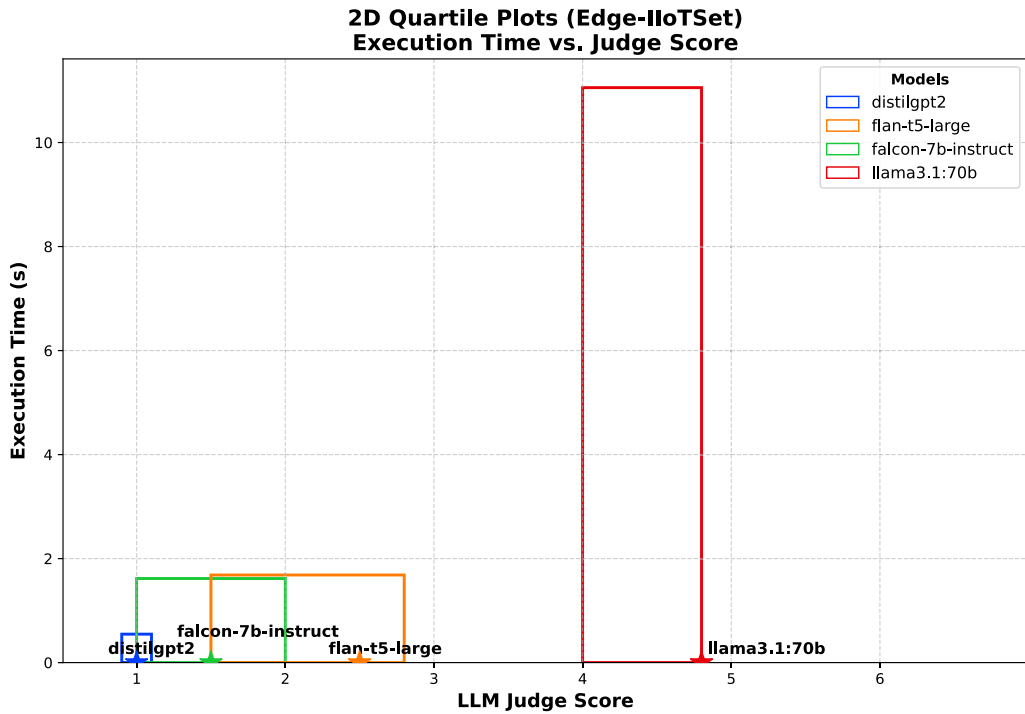
[†]*Note.* Cohen's d is computed as the standardized mean difference between Model A and Model B in each pairwise comparison. A positive value indicates that Model A received a higher mean judge score than Model B, whereas a negative value indicates that Model A received a lower mean judge score than Model B.

cated content. Instead, an RAG-based framework can ground explanations against trusted protocol specifications and attack signatures before the final generation, thereby improving forensic reliability in Tier-1 SOC environments.

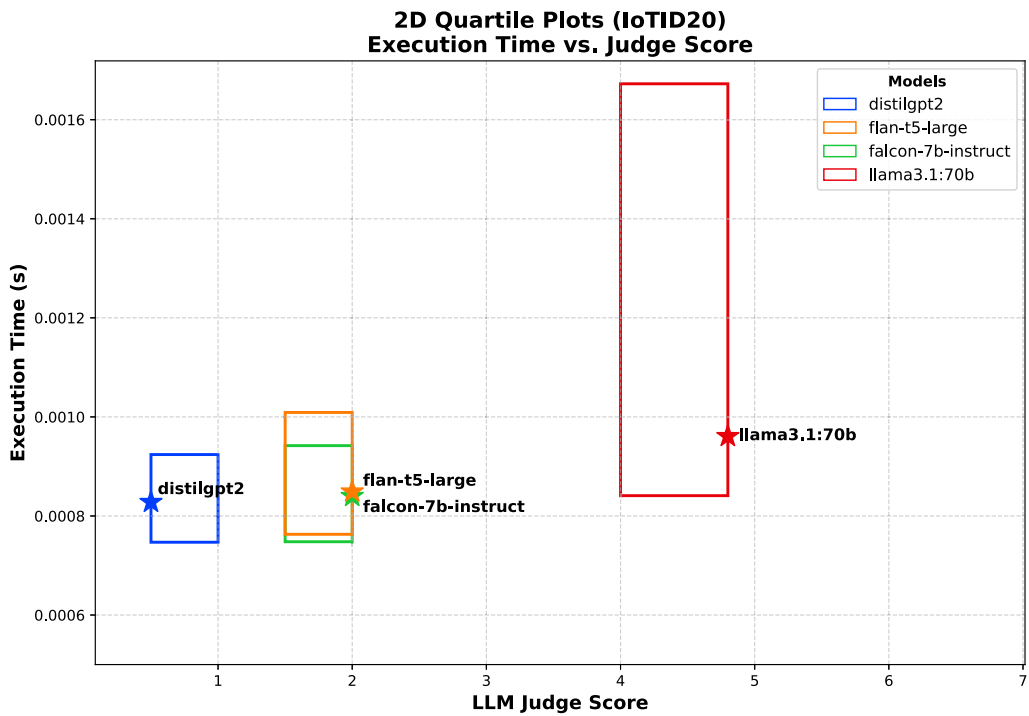
A counterintuitive finding was the latency behavior of `flan-t5-large` relative to `falcon-7b-instruct`. Despite having approximately 9× fewer parameters (780M vs. 7B), `flan-t5-large` exhibited higher execution times on both datasets. For IoTID20, it averaged 0.577s compared to 0.400s for `falcon-7b`. This difference stems from the architectural and decoding strategies. `flan-t5` uses an encoder-decoder architecture that requires a computationally intensive encoding stage before generation, particularly for verbose NIDS feature descriptions. In contrast, `falcon-7b` is a decoder-only model that benefits from optimized prompt-processing mechanisms such as FlashAttention. Furthermore, T5-based models often rely on beam search, which requires multiple parallel forward passes per token, whereas Falcon typically employs faster greedy or sampling-based decodings.

Finally, the results across the two benchmarks reveal a clear sensitivity to the problem space dimensions. The execution times were universally higher for Edge-IIoTSet ([Fig. 10\(a\)](#)) than for IoTID20 ([Fig. 10\(b\)](#)). This increase in latency is attributable to the significantly higher cardinality of the target classes in Edge-IIoTSet, which requires the model to distinguish among 15 distinct classes (including specific attacks such as XSS, SQL injection, and Ransomware) compared to only 5 classes in IoTID20. Furthermore, while Edge-IIoTSet utilizes fewer features (46 vs. 79), the nature of its features (which include verbose protocol-specific fields such as `mqtt.topic`, `dns.qry.name`, and `http.referer`) likely requires a higher token density per feature than the primarily statistical numerical flow metrics (e.g., `Flow_IAT_Mean`) used in IoTID20.

Hallucination Sensitivity to Attack Classes and Entropy Conditions. Analyzing hallucination rates across ground-truth attack classes and actual entropy bins reveals heavy class dependency ([Fig. 11](#)). The Edge-IIoTSet heatmap selectively displays 10 of the 15 original classes, excluding five minority categories (Normal, Port_Scanning, XSS, Password, and SQL_injection) that lacked statistically significant footprints (< 100 samples) or were predicted with such high base confidence that they never exceeded the entropy threshold. Among the 10 evaluated classes, subtle application-layer attacks (e.g., Ransomware) and complex behaviors (e.g., DDoS_ICMP) consis-



(a) Edge-IIoTSet Dataset



(b) IoTID20 Dataset

Fig. 10. 2D Quartile analysis of LLM performance. The plots visualize the trade-off between the execution time (y-axis) and LLM judge score (x-axis) for (a) Edge-IIoTSet and (b) IoTID20. The boxes represent the interquartile range (IQR), and the stars indicate the median values.

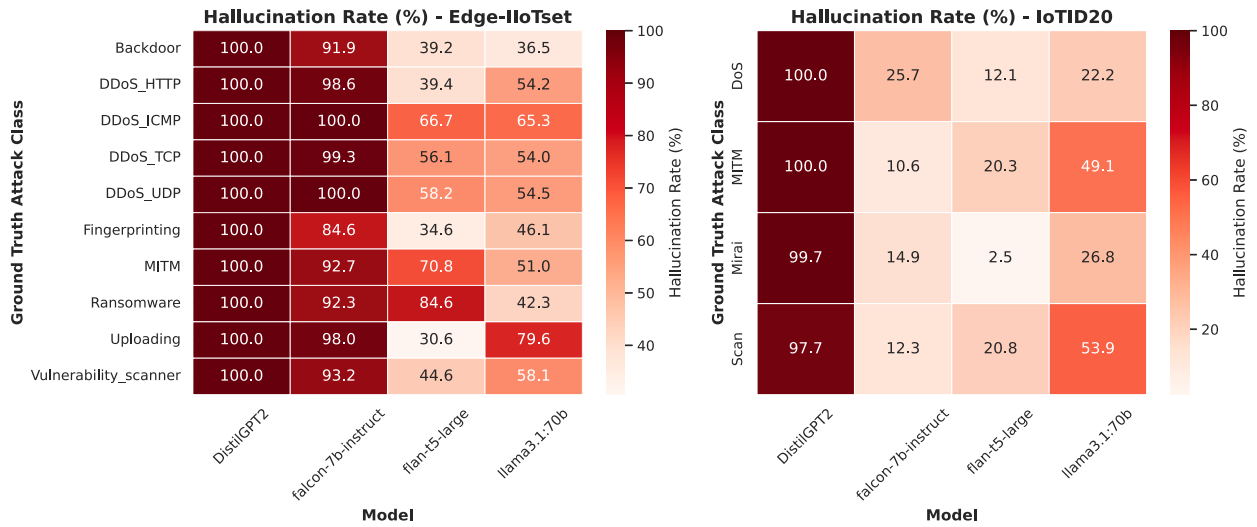


Fig. 11. Hallucination rate (%) across ground truth attack classes by model and dataset. To ensure robust statistical representation, the Edge-IIoTSet visualization selectively reports principal threat categories, excluding minority classes with fewer than 100 evaluated instances.

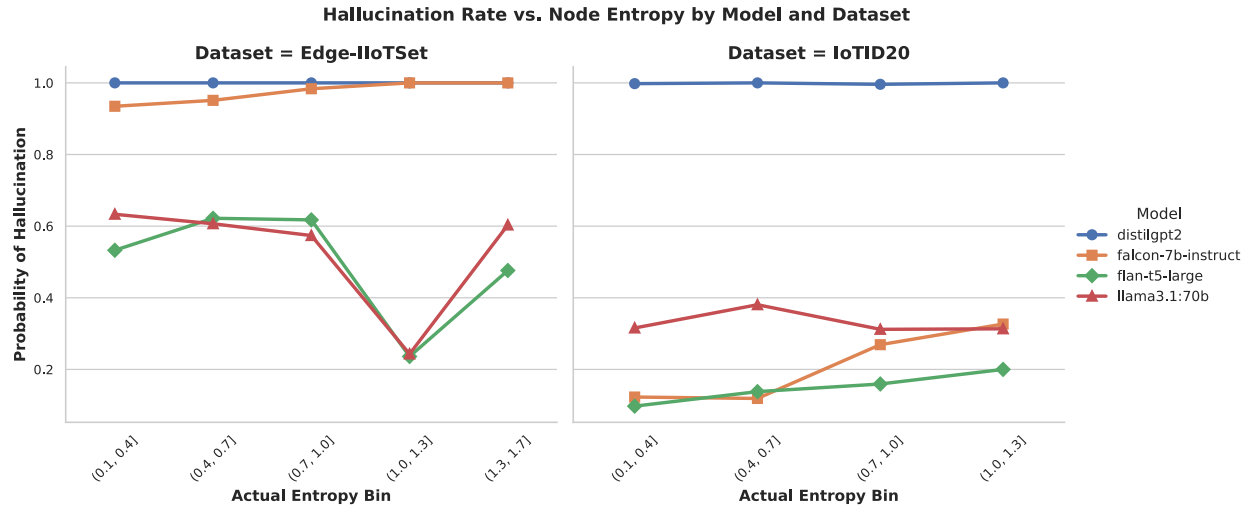


Fig. 12. Hallucination rate vs. actual node entropy by model and dataset.

tently drove severe hallucination rates. Because malicious payloads are executed within data packets, they leave minimal statistical traces at the network flow level. Consequently, the lack of fine-grained payload cues forces LLMs to over-extrapolate from subtle noise, fabricating reasoning to bridge the semantic gap between benign-looking flow metrics and a malicious alert [21,32].

Conversely, volumetric attacks (DDoS_HTTP, DDoS_UDP) yield lower hallucination rates in larger models. These vectors exhibit massive, unambiguous statistical deviations (extreme packet rates and byte counts) that provide definitive causal anchors aligned with pre-trained semantic concepts of a network flood, minimizing the need to fabricate context [19].

IoTID20 data further supports this dichotomy. Capable models achieve a 0% hallucination rate on Normal traffic but struggle severely with stealthy attacks like Scan (53.9% hallucination rate for Llama-3.1-70B) and MITM (49.1%). Reconnaissance and spoofing camouflage within the baseline traffic yielded a low statistical variance. Explaining these stealthy classes without distinct causal evidence frequently forces models to hallucinate technical details to fulfill the generation prompt [30,31].

Mapping base prediction entropy against hallucination risk (Fig. 12) reveals a clear positive correlation for mid-tier models (Falcon-7B-Instruct, Flan-T5-Large) on IoTID20, confirming base ambiguity directly fuels fabrication. Interestingly, Edge-IIoTSet displayed a nonlinear trend, where hallucinations dropped sharply in the highly uncertain (1.0, 1.3] entropy bin for Llama-3.1-70B and Flan-T5-Large. This implies that certain ambiguous flows retain distinct feature combinations that confuse the statistical GNN but are recognized from LLM pre-training corpora, enabling factual generation despite classifier uncertainty. Meanwhile, the ultra-lightweight DistilGPT2 fails universally, maintaining near 100% hallucination rates regardless of entropy.

Table 15

Aggregated Guardrail performance: hallucination catch rate vs. false positive rate by dataset and model.

Dataset	Model	Total Invocations	Judge Hallucinations	Judge Valid	Catch Rate (TPR)	False Positive Rate
Edge-IIoTSet	DistilGPT2	232	232	0	100.0%	N/A [†]
	google/flan-t5-large	232	135	97	14.8%	9.3%
	ollama:llama3.1:70b	232	33	199	27.3%	13.1%
	tiituae/falcon-7b-instruct	232	221	11	53.4%	18.2%
IoTID20	DistilGPT2	362	358	4	100.0%	100.0%
	google/flan-t5-large	362	44	318	56.8%	46.5%
	ollama:llama3.1:70b	362	7	355	42.9%	50.4%
	tiituae/falcon-7b-instruct	362	42	320	59.5%	54.1%

[†] FPR is undefined when the number of judge-valid explanations is zero. Therefore, the DistilGPT2 Edge-IIoTSet FPR should be interpreted as N/A rather than 0%. A 100% catch rate with no valid outputs only indicates complete suppression of hallucinated outputs, not useful discrimination between valid and invalid narratives.

Note. This table evaluates the deterministic hallucination guardrail on the guardrail-assessed subset of the LLM outputs. The columns “Judge Hallucinations” and “Judge Valid” define the reference labels within this subset. Therefore, the ratio of judge hallucinations to total invocations in this table should not be interpreted as the same pipeline-level hallucination rate reported in Tables 12 and 13.

Mitigation: Deterministic Output Verification. While quantifying hallucination rates provides insight into model reliability, deploying LLMs in security-critical SOC requires robust, deterministic guardrails. To mitigate the risks of semantic hallucination, we implemented and evaluated an Output Verification Guardrail with Confidence-Gated Suppression.

Guardrail Architecture: Because the LLM generation in the NEXUS-IDS framework is explicitly anchored to GraphLIME attributions, we implemented a lightweight, post-hoc verification script. Immediately after the LLM generates a narrative, the script uses regular expressions to extract all technical network feature tokens (e.g., `http.content_length` and `Flow.Pkts/s`) from the text. Crucially, the script cross-references these extracted tokens with the actual Top-10 GraphLIME features, which are computationally derived for that specific network flow. To prevent aggressive over-filtering, the script uses a dynamic exclusion list to ignore standard networking acronyms (e.g., TCP/IP) and missing value indicators (e.g., N/A) commonly used by larger foundation models. If the LLM fabricated a feature not present in the authorized GraphLIME list or fell into a degenerate repetition loop, the system immediately suppressed the narrative and fell back to a safe, deterministic template (for example, “[SUPPRESSED] The system detected a semantic hallucination. Prediction was made based strictly on GraphLIME features: [Features]”).

Guardrail Efficacy and Fluency Bias: Table 15 details the performance of the guardrail across both datasets. The DistilGPT2 results illustrate a degenerate boundary case rather than a success: on Edge-IIoTSet the model produced zero judge-valid explanations, so although the guardrail flagged 100% of hallucinations, its FPR is undefined (N/A) rather than a genuine 0% (Table 15). On IoTID20 the same model yielded a 100% catch rate together with a 100% FPR over only four valid outputs, meaning that the guardrail suppressed both hallucinated and valid narratives indiscriminately. Both outcomes reflect DistilGPT2’s near-total generation failure (Tables 12 and 13) rather than effective discrimination, and the guardrail metrics are therefore not meaningfully interpretable for this model. For the models that produce a non-trivial mix of valid and invalid outputs, the evaluation revealed a significant performance disparity on the IoTID20 dataset, with the mid-tier and foundation models exhibiting FPRs ranging from 46.5% to over 50%.

Crucially, a manual review of these “False Positives” highlighted a potential limitation in standard LLM-as-a-Judge evaluation frameworks. In several instances, foundation models (such as Llama-3.1-70B) generated highly fluent narratives that subtly incorporated technical features beyond the Top-10 GraphLIME attributions for the target flow. The Mixtral-8x22B judge frequently evaluated these explanations as “Valid,” suggesting a vulnerability to *fluency bias*, which is the tendency of evaluator LLMs to overlook subtle technical inaccuracies when the surrounding text is grammatically and logically coherent.

While our deterministic guardrail computationally flagged these unauthorized feature insertions, we acknowledge that this strict regex-based suppression serves primarily as an initial, proof-of-concept mitigation strategy. Although a portion of the elevated FPR on the IoTID20 dataset reflects instances where the deterministic guardrail enforced stricter feature adherence than the LLM evaluator, its rigidity also heavily penalizes benign phrasing variations. This finding underscores the fact that while structural anchoring is valuable for operational security, further investigation is required. Future work should explore more elaborate mitigation techniques to achieve a nuanced balance between strict fidelity and narrative flexibility. Promising avenues include semantic embedding-based verification (to tolerate benign syntactic variations of authorized features), inference-time constrained decoding mechanisms (e.g., regex-guided beam search), and multi-agent reflection pipelines where the LLM iteratively critiques and self-corrects its own output against the GraphLIME constraints prior to final generation.

6.4.3. Ablation studies of the semantic layer

To comprehensively evaluate the individual contributions of our proposed semantic mechanisms, we conducted targeted ablation studies focusing on two distinct dimensions: operational efficiency (pipeline components) and explanation reliability (warning injection).

Operational Pipeline Components (Cache and Filter). To evaluate the individual contributions of the semantic cache and the entropy-based filter, we conducted an ablation study across all candidate models. Table A.4 and Table A.5 (Appendix A) present the execution

time and explanation fidelity metrics under three configurations: the complete pipeline (*Full*), the pipeline with the semantic cache disabled (*No semantic caching*), and the pipeline with the low-entropy bypass disabled (*No entropy filter*).

The results clearly demonstrate that the semantic cache is the primary driver of computational efficiency. As shown in Table A.5 (Appendix A), removing the cache (*No semantic caching*) drastically increased the mean inference time for the llama3.1:70b model from 0.62s to 4.34s. Furthermore, without the cache, the Time IQR jumps to 12.13s, exposing the system to severe tail-latency spikes that would cripple real-time monitoring. Crucially, this massive reduction in processing time under the *Full* mode is achieved without sacrificing explanation fidelity; the llama3.1:70b mean score remains exceptionally high (4.65 vs. 4.68) regardless of cache utilization.

Similarly, disabling the entropy filter (*No entropy filter*) forces the LLM to generate explanations for easily classified, routine alerts, unnecessarily taxing the system. As shown in Table A.4 (Appendix A), evaluating llama3.1:70b without the entropy filter more than doubles the mean execution time compared to the *Full* system (1.69s vs. 0.71s). Forcing the LLM to explain all samples occasionally yields minor score improvements for mid-tier models such as falcon-7b-instruct; however, this marginal gain does not justify the disproportionate increase in computational overhead. Ultimately, the *Full* pipeline configuration successfully harmonizes real-time operational efficiency with high-fidelity semantic intelligence.

Impact of Entropy-Aware Warning Injection. To directly validate the core claim of the entropy-aware calibration mechanism, we conducted a targeted ablation study comparing hallucination rates with and without the cautionary warning injection. We extracted a 5% random sample of network flows from both datasets, isolating only the high-entropy predictions ($H > 0.3$) that triggered the LLM semantic layer. In the control configuration, the LLM was prompted to explain the alert without uncertainty modifiers. In the treatment configuration, the prompt dynamically injected a mandatory cautionary warning regarding the model's low confidence. All responses were subsequently evaluated using the identical LLM-as-a-Judge hallucination criteria.

The comparative results of this control-versus-treatment experiment show that, globally across the 400 evaluated samples, the warning injection yielded a modest overall hallucination reduction of 4%. However, a model-specific breakdown reveals a significant architectural divergence in how LLMs handle forced uncertainty.

For mid-tier instruction-tuned models, the warning acts as a highly effective grounding mechanism. Specifically, flan-t5-large achieved hallucination reductions of 36% on IoTID20 and 8% on Edge-IIoTSet, whereas falcon-7b-instruct experienced a 34% reduction on IoTID20 (despite a minor performance degradation of 8% on Edge-IIoTSet). By explicitly conditioning these models on their uncertainty, the prompt successfully suppresses their tendency to fabricate false narratives to fill semantic gaps. The ultra-lightweight DistilGPT2 showed no change (0% improvement), failing universally under both conditions due to its inability to process the complex feature arrays.

Conversely, the foundation model (llama3.1:70b) exhibited a paradoxical degradation under the treatment condition, with hallucination rates increasing by 45% on IoTID20 and 17% on Edge-IIoTSet. Our qualitative review indicates that advanced foundation models exhibit strong intrinsic calibration of uncertainty. Forcing a rigid, rule-based cautionary preamble disrupts the natural reasoning flow, occasionally causing the model to over-explain, invent caveats, or fabricate technical scenarios to logically justify the forced uncertainty. Consequently, while explicit warning injection is a vital guardrail for lightweight and mid-tier NIDS deployments, foundation models perform more factually when allowed to navigate structural uncertainty natively.

6.5. Structured human expert evaluation and judge validation

To validate the operational utility and factual reliability of the generated narratives, and to explicitly calibrate and verify the automated LLM-as-a-Judge assessments, we conducted a structured human-in-the-loop evaluation.

Random Extraction and Evaluation Setup. A random sample of 96 LLM invocations was extracted from the test sets. To ensure balanced coverage across architectures and environments, the sample consisted of 12 unique flow nodes for each of the two datasets (Edge-IIoTSet and IoTID20), evaluated across all four candidate LLMs (12 nodes $\times$ 2 datasets $\times$ 4 models = 96 explanations). Four independent cybersecurity experts blindly evaluated the sampled explanations. For each instance, the experts were provided with raw network features, the IDS-FN-GNN predicted label, prediction entropy, top GraphLIME attributions, and the generated semantic narrative. They evaluated two metrics: a decimal Quality Score ranging from 0.0 (Very Poor) to 5.0 (Excellent) based on clarity, context utilization, and uncertainty communication; and a binary Hallucination Flag (Yes/No) to identify fabricated features, incorrect protocols, or logically inconsistent claims.

Inter-Annotator Agreement (IAA). To verify the reliability of the human assessment across the 96 strictly matched instances, we computed the Inter-Annotator Agreement. For the numerical quality ratings (0–5), the Intraclass Correlation Coefficient (ICC2,1) was 0.91, indicating excellent absolute agreement among the four experts. For binary hallucination detection, Fleiss' kappa was 0.41, representing moderate agreement. This moderate Kappa reflects the inherently subjective boundaries of defining a “hallucination” in complex network traffic, where experts may diverge on whether a vague statement constitutes a factual fabrication. Nevertheless, the exceptionally high ICC confirms a strong consensus regarding the overall utility and quality of the narratives.

Validation of the LLM-as-a-Judge (Calibration, Bias, and Consistency). A critical objective of this human evaluation was to address the inherent limitations of relying solely on an automated evaluator (Mixtral-8x22B), specifically regarding self-evaluation bias, score calibration, and inter-rater consistency. To anchor the 0–5 scoring scale to human validation and assess potential architectural bias

Table 16

Human expert evaluation: mean explanation quality and hallucination rates. Results are aggregated across 4 experts evaluating 96 samples (12 nodes $\times$ 2 datasets $\times$ 4 models), providing a human-anchored baseline for the automated LLM-as-a-judge metrics.

Dataset	Model	Mean Score (0–5)	Std Dev	Hallucination Rate (%)
Edge-IIoTSet	Llama-3.1-70B	4.2	0.5	2.1
	Flan-T5-Large	1.7	0.3	0.0
	Falcon-7B-Instruct	1.6	0.4	60.4
	DistilGPT2	0.3	0.2	77.1
IoTID20	Llama-3.1-70B	3.9	0.7	10.4
	Falcon-7B-Instruct	1.6	0.4	41.7
	Flan-T5-Large	1.5	0.5	14.6
	DistilGPT2	0.3	0.2	77.1

(e.g., Mixtral systematically favoring large decoder-only models), we compared the automated metrics to the human ratings. As shown in Table 16, the human expert evaluation broadly confirms the main trend observed in the automated assessment: Llama-3.1-70B achieved the highest perceived explanation quality, whereas DistilGPT2 produced operationally unusable outputs. However, the mid-tier models, Flan-T5-Large and Falcon-7B-Instruct, were close in human quality scores, and their ordering was dataset-dependent. Therefore, human evaluation should be interpreted as validating the broad separation between high- and low-capacity models rather than confirming an exact model hierarchy across all datasets. The human scores were also generally stricter than the automated judge, particularly for mid-tier models, indicating that automated scores are useful comparative indicators but may overestimate absolute operational utility compared with expert SOC analysts. Finally, to quantify the judge's internal consistency, we conducted a repeated-measures test on a 5% random subset (119 samples), evaluating each explanation five independent times. At our selected operational temperature of 0.1, the judge demonstrated extreme determinism: 90.8% of evaluations yielded perfectly identical scores across all five runs, resulting in a negligible mean variance of 0.0085. A comparative baseline at a higher temperature ($T=0.6$) reduced the perfect consistency to 43.7% and increased the mean variance by nearly tenfold (0.0912). Furthermore, the global average score remained entirely stable across both temperatures (2.65 versus 2.67), proving the automated evaluations are highly reproducible and immune to temperature-induced score drift.

Results and Assessment of Hallucinations. Table 16 presents the aggregated human evaluation results. Llama-3.1-70B achieved mean expert quality scores of 4.2 and 3.9 for the Edge-IIoTSet and IoTID20 datasets, respectively. Crucially, a comparison of hallucination rates revealed a major divergence between automated and human assessments. While the automated judge flagged Llama-3.1-70B with high hallucination rates (54.85% on Edge-IIoTSet and 34.26% on IoTID20), human experts flagged only 2.1% and 10.4% of its responses as containing operational hallucinations. This discrepancy stems from the strict, literal feature-matching criteria used by the LLM judge (which penalized benign semantic or syntactic variations in feature names), whereas human experts evaluated the overarching factual correctness of the network behavior being described.

Conversely, the smaller foundation models exhibited severe, universally recognized failure modes. DistilGPT2 failed to generate coherent explanations, yielding near-zero human-quality scores (~ 0.3) and hallucinating in more than 77% of its responses. Flan-T5-Large exhibited a “safe but useless” mode; while it rarely hallucinated on the Edge-IIoTSet (0.0%), it generated highly templated text lacking actionable context. Falcon-7B struggled to bridge the semantic gap, frequently fabricating technical definitions for network ports, resulting in expert-flagged hallucination rates of 41.7%–60.4%. Ultimately, human validations corroborate the core premise derived from the automated metrics: while the automated judge serves as a highly conservative upper bound for strict template adherence, human evaluation confirms that translating complex IDS alerts into trustworthy, operationally faithful narratives strictly requires the reasoning capacity of larger foundation models.

7. Key findings and limitations

This section summarizes the principal contributions and empirical outcomes of the NEXUS-IDS, acknowledges the methodological constraints, and identifies directions for future research.

7.1. Key findings

A comprehensive evaluation identified six principal findings that validated the dual objectives of NEXUS-IDS: accurate detection and transparent explainability. These findings establish benchmarks for LLM-powered semantic interpretation in IoT security. **Detection Performance Excellence.** The IDS-FN-GNN architecture demonstrated robust intrusion detection capabilities across heterogeneous IoT environments. On the Edge-IIoTSet dataset, the model achieves 94.30% accuracy with a macro-F1 of 93.22%, whereas IoTID20 yields 97.42% accuracy with a macro-F1 of 97.41%. In particular, volumetric attacks (DDoS variants) achieve near-perfect detection ($F1 > 99\%$), whereas more subtle application-layer attacks (SQL Injection, XSS, Fingerprinting) maintain F1-scores between 83–88%, demonstrating balanced performance across attack sophistication levels. **Effectiveness of Flow-Node Graph Representation.** The bipartite-like graph construction, where flows are modeled as nodes connected through shared device endpoints, enables

Table 17

Tiered deployment topology of NEXUS-IDS with minimum hardware requirements per tier.

Tier	Components	Minimum Hardware
IoT Endpoint	None (protected asset)	Unmodified
Edge Gateway	IDS-FN-GNN, GraphLIME	4-core CPU, 8 GB RAM, 1 Gbps NIC
Regional Edge (optional)	Flan-T5-Large / Falcon-7B-Instruct	1× GPU 8–24 GB VRAM, 32 GB RAM
Centralized SOC	Llama-3.1-70B (Q4), cache, dashboard	1× A100 80 GB, 128 GB RAM

effective two-hop relational learning. This architecture captures coordinated attack patterns, such as botnets and scanning campaigns, which manifest across multiple flows that share common communication endpoints. These results support the hypothesis that graph-based representations provide superior contextual awareness compared with flow-independent approaches. **Structural Explainability Validation.** GraphLIME successfully identified semantically meaningful features that drive model predictions. For DDoS attacks, protocol-specific attributes (UDP stream characteristics, ICMP checksums, and TCP connection states) are dominant contributors, whereas scanning activities are characterized by temporal features (SYN flag counts, flow duration, and inter-arrival statistics). The alignment between the identified features and known attack signatures confirms that the model decisions are grounded in legitimate traffic semantics rather than spurious correlations. **LLM Quality-Complexity Trade-offs.** A systematic evaluation across four LLM architectures revealed a clear quality hierarchy at the extremes: Llama-3.1-70B achieved the highest semantic quality scores, whereas DistilGPT2 produced operationally unusable outputs. Flan-T5-Large and Falcon-7B-Instruct formed a mid-tier group whose relative ordering varied across datasets and evaluation settings, indicating that mid-sized model performance is sensitive to dataset complexity and generation behavior. **Operational Efficiency of Entropy-Based Filtering.** The entropy-aware caching mechanism constrains LLM invocations to approximately 5% of the samples across all tested entropy thresholds ($\tau \in \{0.1, 0.3, 0.5\}$). The cache hit rates range from 10 to 34% depending on the dataset characteristics and threshold stringency. This demonstrates that the architecture reserves computationally expensive reasoning for genuinely ambiguous cases while leveraging the cached explanations for recurring patterns. **Quality-Reliability Divergence Across Model Scales.** A non-monotonic relationship was observed between the model size and factual accuracy. Although Llama-3.1-70B achieved the highest quality scores, it exhibited moderate hallucination rates (34–55%), whereas the smaller Flan-T5-Large demonstrated the lowest hallucination rates (14–53%) despite lower quality scores. These findings indicate that perceived explanation quality and factual reliability are distinct dimensions that require independent evaluations in security-critical applications. **Tiered Deployment Feasibility.** The empirical profiling across model scales validates a practical three-tier deployment topology in which no NEXUS-IDS component executes on IoT endpoints themselves; devices remain unmodified and are observed passively at network aggregation points. As summarized in Table 17, the deployment topology is organized as follows:

- **IoT Endpoint Tier:** No NEXUS-IDS components execute here. Devices remain unmodified and are observed passively, incurring zero computational or energy overhead.
- **Edge Gateway Tier:** Hosts the IDS-FN-GNN detector and GraphLIME explainer, processing 100% of flows in real time on commodity hardware (4–8 core CPU, 8–16 GB RAM, no GPU mandatory). The two-layer GNN with hidden dimension 64 and the sparse local GraphLIME surrogates impose a minimal computational footprint.
- **Regional Edge Tier (optional):** May host mid-range models (Flan-T5-Large or Falcon-7B-Instruct on 8–24 GB VRAM GPUs with 32 GB system RAM) to reduce round-trip latency in geographically distributed sites.
- **Centralized SOC Tier:** Hosts the foundation LLM (Llama-3.1-70B, 4-bit quantized, ~40 GB VRAM on a single A100 with 128 GB system RAM), the global semantic cache, and the analyst dashboard. Because entropy filtering ($\tau = 0.3$) and semantic caching jointly restrict foundation-LLM invocations to $\approx 5\%$ of flows, a single A100-class accelerator sustains explanation throughput for enterprise-scale deployments.

This topology demonstrates that the tension between LLM resource demands and IoT constraints can be reconciled by deploying each component at the tier where its cost-benefit profile is justified. Detection and structural explainability remain fully functional on edge hardware, independent of the SOC tier.

7.2. Limitations

Despite its strong empirical performance, the proposed NEXUS-IDS framework exhibits several limitations, which are discussed below. **Stealthy-attack detection limitations.** The lower performance on application-layer stealthy attacks is mainly due to sparse graph neighborhoods and minority-class under-representation, which limit the current flow-as-node design. Notably, errors occurred mostly among stealthy classes rather than between stealthy and Normal traffic, which is operationally more favorable. However, these classes also exhibit a higher semantic hallucination risk, creating a compounding limitation in which the most difficult attacks to classify may also be the hardest to explain reliably. Possible future improvements include temporal flow aggregation, payload-aware edge features, attention-based aggregation for sparse neighborhoods, ensemble integration with a flow-level classifier, and class-balanced or focal-loss training, all without affecting the dual-layer explainability design of NEXUS-IDS. **LLM-as-a-Judge Framework Biases.** Although the LLM-as-a-judge framework offers a scalable alternative to human annotation, it introduces specific biases that must be acknowledged. Research has extensively documented verbosity bias, in which judge models disproportionately favor longer, more structured responses regardless of their underlying factual superiority. This vulnerability warrants careful examination within

Table 18

Verbosity bias analysis: correlation between explanation length and LLM judge score by dataset and model.

Dataset	Model	Avg Score	Pearson (r)	Spearman (ρ)
Edge-IIoTSet	DistilGPT2	1.05	-0.1962	-0.0898
	flan-t5-large	3.30	0.3792	0.3216
	llama3.1:70b	4.80	0.0249	-0.0144
	falcon-7b-instruct	2.54	-0.4280	-0.3697
IoTID20	DistilGPT2	0.98	-0.1496	-0.3119
	flan-t5-large	2.50	-0.0653	-0.0551
	llama3.1:70b	4.69	-0.0309	-0.0592
	falcon-7b-instruct	2.68	-0.1422	-0.0288

our results; for instance, as shown in Table A.3 (Appendix A), llama3.1:70b consistently achieves near-perfect scores (4.8) while generating lengthy, highly formatted warnings, raising the potential concern that its evaluations might be artificially inflated by response length.

To strictly quantify the extent of this verbosity bias within our evaluation, we calculated the correlation between the explanation word count and the assigned judge score. While the global baseline across all invocations exhibited a weak positive correlation (Pearson $r = 0.32$), isolating these metrics by specific models and datasets (Table 18) revealed that the length-based bias was statistically negligible. Crucially, for the highest-performing model (llama3.1:70b), the correlation was effectively zero ($r \approx 0.02$), indicating that its near-perfect scores were driven by factual accuracy rather than response length. Furthermore, for models that are more prone to ungrounded rambling (e.g., falcon-7b-instruct), the correlation is notably negative (e.g., $r = -0.43$ on the Edge-IIoTSet). This demonstrates that when verbosity resulted in the hallucination of unauthorized features, the evaluator successfully penalized this output. Consequently, while automated evaluation frameworks are generally susceptible to verbosity biases, our strict criteria-driven prompt ensured that scores predominantly reflected factual adherence to the structural GraphLIME attributions.

Furthermore, while judges can exhibit self-preference bias [40], we mitigated this risk by employing the Mixtral-8x22B as the evaluator. Its sparse Mixture-of-Experts (MoE) architecture is architecturally distinct from the dense transformer architectures of the four generator models (DistilGPT2, Flan-T5, Falcon, and Llama-3), thereby minimizing architectural alignment bias. Despite this structural decoupling, intra-rater inconsistency remains an observable limitation; this instability is evident in the disparate scores assigned to flan-t5-large (ranging from 2.5 to 4.8) for producing nearly identical feature list explanations across the IoTID20 and Edge-IIoTSet datasets. Consequently, while automated scores provide a useful relative ranking of reasoning capability, they should be interpreted as proxies for linguistic coherence rather than absolute measures of forensic accuracy. Similarly, the hallucination rates reported in Section 6.4.2 should be interpreted as approximate indicators derived from automated evaluation rather than definitive ground-truth measurements. **Latency Constraints for Real-Time Deployment.** Although per-component latency measurements reported in Sections 6.2.5 and 6.3.4 confirm that IDS-FN-GNN detection (0.034 μ s/flow in batch mode, 3.86 ms in online mode) and GraphLIME attribution (9.48 ms/flow, p95 = 16.97 ms) are computationally viable on the GPU-equipped workstation used in our experiments, this study does not provide a dedicated end-to-end latency benchmark on representative constrained edge hardware (e.g., NVIDIA Jetson, ARM-based gateway) under live traffic conditions. A full pipeline characterization, including concurrent flow arrival and incremental graph update overheads, remains an important direction for future work. **Static Entropy Threshold Configuration.** The current framework employs fixed entropy thresholds ($\tau = 0.3$ by default) across all attack classes and operational contexts. However, optimal thresholds may vary by attack type. For instance, stealthy attacks with ambiguous signatures may require lower thresholds than volumetric attacks with clear patterns. Adaptive threshold mechanisms responsive to class-specific uncertainty distributions remain unexplored. **Limited Scope of Human-Subject and Live-SOC Validation.** While we conducted a structured human expert evaluation to validate explanation quality, factual grounding, and hallucination rates (Section 6.5), this assessment is based on a limited sample of 96 explanations evaluated by four experts. This small-scale evaluation provides preliminary evidence of narrative utility and factual reliability, but it is not a substitute for a comprehensive human-subject study. Furthermore, this study did not evaluate the framework within a live Security Operations Center (SOC) workflow. Therefore, the ultimate impact of dual-layer explanations on analyst decision-making speed, accuracy, and operational trust calibration in high-pressure environments remains to be fully characterized. To address these gaps, our follow-up work will conduct a larger, longitudinal human-subject evaluation involving Tier-1 and Tier-2 SOC analysts in a simulated live-alert environment, comparing task completion times and cognitive load against baseline black-box alerts. **Computational Resource Requirements.** The complete NEXUS-IDS pipeline, including GNN inference, GraphLIME explanation generation, and LLM semantic translation, requires substantial computational resources (NVIDIA A100 GPUs for the LLM experiments). Deployment in resource-constrained edge environments may necessitate model distillation, quantization, hybrid cloud-edge architectures, or on-device language models, which were not explored in this study. **Assistive Prototype Framing and Deployment Considerations.** While foundation models demonstrate significant potential in translating technical metrics into intuitive narratives, the hallucination rates observed in our evaluation (e.g., ~55% for the Llama-3.1-70B model on Edge-IIoTSet) and the current false-positive rates of deterministic guardrails indicate that the semantic layer is best positioned as an assistive research prototype. In a dynamic SOC environment, managing these hallucination rates is important to maintain optimal analyst trust calibration and support precise incident responses, particularly for stealthy threats. To maximize the utility of the framework, we recommend that LLM-generated narratives serve as contextual aids rather than as standalone evidence for automated mitigation.

Raw network telemetry and deterministic GraphLIME attributions are the primary foundations for verifying network anomalies. Consequently, the practical integration of these models is most effective in an HITL workflow. In this collaborative setup, LLM outputs offer valuable high-level hypotheses that seamlessly guide and support secondary verification by human analysts.

8. Conclusion

This study introduces NEXUS-IDS, a unified framework that integrates graph-based deep learning, XAI, and LLMs for transparent IoT intrusion detection. The evaluation of the Edge-IIoTSet and IoTID20 datasets demonstrated robust detection performance across heterogeneous attack categories with human-centric explainability.

The core contributions of the framework, which are the flow-node GNN architecture, dual-layer explainability combining GraphLIME with LLM semantic translation, and entropy-aware uncertainty calibration with hallucination detection, establish a new paradigm for trustworthy AI-driven security. The entropy-based caching mechanism enables efficient real-time deployment by limiting LLM invocations to high-uncertainty samples while maintaining explanation quality and factual grounding.

NEXUS-IDS addresses the critical requirements for deployment in high-stakes environments, including regulatory compliance, verifiable explanations, and transparency regarding uncertainty. However, we explicitly caution that while the core graph-based detection and structural attribution components are robust, the LLM-driven semantic layer remains a research prototype. Because hallucination rates remain unmitigated at operational scales, deploying this semantic layer currently mandates strict HITL oversight to ensure analysts are not misled by fluent but fabricated security narratives. By demonstrating that accuracy, technical explainability, and human-centered understanding can be achieved synergistically, the framework establishes new benchmarks and identifies the remaining reliability hurdles for transparent and actionable AI-driven security analytics in next-generation IoT ecosystems.

Future work will focus on (1) adaptive real-time learning for evolving threats, (2) human-subject studies quantifying the impact on analyst decision-making, (3) automated explanation quality metrics, (4) cross-domain generalization to additional IoT verticals, and (5) domain-specific LLM fine-tuning for specialized security terminology.

CRedit authorship contribution statement

Maha Driss: Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Safa Ben Atitallah:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Adel Ammar:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology; **Wadii Boulila:** Supervision, Project administration, Methodology, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors gratefully acknowledge the support of Prince Sultan University.

Data availability

Data will be made available on request.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at [10.1016/j.iot.2026.102010](https://doi.org/10.1016/j.iot.2026.102010)

References

- [1] Y.B. Zikria, H. Yu, M.K. Afzal, M.H. Rehmani, O. Hahm, Internet of things (IoT): operating system, applications and protocols design, and validation techniques, *Future Gener. Comput. Syst.* 88 (2018) 699–706. <https://doi.org/10.1016/j.future.2018.07.058> <https://www.sciencedirect.com/science/article/pii/S0167739X18317710>
- [2] Y. Li, Y. Zuo, H. Song, Z. Lv, Deep learning in security of internet of things, *IEEE Internet Things J.* 9 (22) (2021) 22133–22146.
- [3] U. Tariq, I. Ahmed, A.K. Bashir, K. Shaukat, A critical cybersecurity analysis and future research directions for the internet of things: a comprehensive review, *Sensors* 23 (8) (2023) 4117.
- [4] S.A. Rahman, H. Tout, C. Talhi, A. Mourad, Internet of things intrusion detection: centralized, on-device, or federated learning?, *IEEE Netw.* 34 (6) (2020) 310–317.
- [5] F.S. Alrayes, M. Zakariah, M. Driss, W. Boulila, Deep neural decision forest (DNDF): a novel approach for enhancing intrusion detection systems in network traffic analysis, *Sensors* 23 (20) (2023) 8362.
- [6] S. Arisdakessian, O.A. Wahab, A. Mourad, H. Otrouk, M. Guizani, A survey on IoT intrusion detection: federated learning, game theory, social psychology, and explainable AI as future directions, *IEEE Internet Things J.* 10 (5) (2022) 4059–4092.
- [7] X. Li, J. Wang, Z. Yan, Can graph neural networks be adequately explained? a survey, *ACM Comput. Surv.* 57 (5) (2025) 1–36.

- [8] Q. Huang, M. Yamada, Y. Tian, D. Singh, Y. Chang, Graphlime: local interpretable model explanations for graph neural networks, *IEEE Trans. Knowl. Data Eng.* 35 (7) (2022) 6968–6972.
- [9] M. Ahmad, Q. Riaz, M. Zeeshan, H. Tahir, S.A. Haider, M.S. Khan, Intrusion detection in internet of things using supervised machine learning based on application and transport layer features using UNSW-NB15 data-set, *EURASIP J. Wirel. Commun. Netw.* 2021 (1) (2021) 10.
- [10] B. Ibrahim Hairab, H.K. Aslan, M.S. Elsayed, A.D. Jurcut, M.A. Azer, Anomaly detection of zero-day attacks based on CNN and regularization techniques, *Electronics* 12 (3) (2023) 573.
- [11] S.B. Atitallah, M. Driss, W. Boulila, IDS-GraphMamba: a Markov-enhanced graph mamba framework for real-time intrusion detection in IoMT edge networks, *Comput. Netw.* 275 (2025) 111933 <https://www.sciencedirect.com/science/article/pii/S1389128625008989>. <https://doi.org/10.1016/j.comnet.2025.111933>
- [12] W.W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, M. Portmann, E-GraphSage: a graph neural network based intrusion detection system for IoT, in: *NOMS 2022-2022 IEEE/iFIP Network Operations and Management Symposium*, IEEE, 2022, pp. 1–9.
- [13] M.T. Ribeiro, S. Singh, C. Guestrin, “Why should i trust you?” explaining the predictions of any classifier, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- [14] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, *Adv. Neural Inf. Process. Syst.* 30 (2017) 4765–4774.
- [15] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: generating explanations for graph neural networks, *Adv. Neural Inf. Process. Syst.* 32 (2019) 9240–9251.
- [16] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, *Adv. Neural Inf. Process. Syst.* 33 (2020) 19620–19631.
- [17] V.Z. Mohale, I.C. Obagbuwa, A systematic review on the integration of explainable artificial intelligence in intrusion detection systems to enhancing transparency and interpretability in cybersecurity, *Front. Artif. Intell.* 8 (2025) 1526221.
- [18] N. Moustafa, N. Koroniotis, M. Keshk, A.Y. Zomaya, Z. Tari, Explainable intrusion detection for cyber defences in the internet of things: opportunities and solutions, *IEEE Commun. Surv. Tutor.* 25 (3) (2023) 1775–1807.
- [19] O. Arreche, T. Guntur, M. Abdallah, Xai-ids: toward proposing an explainable artificial intelligence framework for enhancing network intrusion detection systems, *Appl. Sci.* 14 (10) (2024) 4170.
- [20] Y. Yigit, M.A. Ferrag, M.C. Ghanem, I.H. Sarker, L.A. Maglaras, C. Chrysoulas, N. Moradpoor, N. Tihanyi, H. Janicke, Generative AI and LLMs for critical infrastructure protection: evaluation benchmarks, agentic AI, challenges, and opportunities, *Sensors* 25 (6) (2025) 1666.
- [21] P.R.B. Houssel, P. Singh, S. Layeghy, M. Portmann, Towards explainable network intrusion detection using large language models, in: *2024 IEEE/ACM International Conference on Big Data Computing, Applications and Technologies (BDCAT)*, IEEE, 2024, pp. 67–72.
- [22] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, *Adv. Neural Inf. Process. Syst.* 33 (2020) 1877–1901.
- [23] S.B. Atitallah, M. Driss, A. Alsehibani, W. Boulila, Adaptive diffusion Markov-enhanced GCN with LLM explanations for IoT attack detection, *Procedia Comput. Sci.* 270 (2025) 4014–4023.
- [24] K. Shuster, S. Poff, M. Chen, D. Kiela, J. Weston, Retrieval augmentation reduces hallucination in conversation, in: *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021, pp. 3784–3803.
- [25] N. Dziri, S. Milton, M. Yu, O.R. Zaiane, S. Reddy, On the origin of hallucinations in conversational models: is it the datasets or the models?, in: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2022, pp. 5271–5285.
- [26] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y.J. Bang, A. Madotto, P. Fung, Survey of hallucination in natural language generation, *ACM Comput. Surv.* 55 (12) (2023) 1–38.
- [27] D. Li, B. Jiang, L. Huang, A. Beigi, C. Zhao, Z. Tan, A. Bhattacharjee, Y. Jiang, C. Chen, T. Wu, K. Shu, L. Cheng, H. Liu, From generation to judgment: opportunities and challenges of LLM-as-a-judge, in: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, Suzhou, China, 2025, pp. 2757–2791. <https://aclanthology.org/2025.emnlp-main.138/>. <https://doi.org/10.18653/v1/2025.emnlp-main.138>
- [28] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J.E. Gonzalez, I. Stoica, Judging LLM-as-a-judge with MT-bench and chatbot arena, in: A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, S. Levine (Eds.), *Advances in Neural Information Processing Systems*, 36, Curran Associates, Inc., 2023, pp. 46595–46623.
- [29] S. Tan, S. Zhuang, K. Montgomery, W.Y. Tang, A. Cuadron, C. Wang, R.A. Popa, I. Stoica, JudgeBench: a benchmark for evaluating LLM-based judges, 2025, <https://arxiv.org/abs/2410.12784>.
- [30] M. Venkateswarlu, C. Karthik, M. Shivamani, A.S. Narayana Rao, Privacy-preserving graph-based intrusion detection system: a federated learning approach with GNN and XAI, in: *2025 9th International Conference on Inventive Systems and Control (ICISC)*, 2025, pp. 405–412. <https://doi.org/10.1109/ICISC65841.2025.11188133>
- [31] J. Govea, R. Gutierrez, W. Villegas-Ch, A. Maldonado Navarro, Hybrid AI for predictive cyber risk assessment: federated graph-transformer architecture with explainability, *IEEE Access* 13 (2025) 122187–122206. <https://doi.org/10.1109/ACCESS.2025.3588076>
- [32] Y.A. Farrukh, S. Wali, I. Khan, N.D. Bastian, XG-NID: dual-modality network intrusion detection using a heterogeneous graph neural network and large language model, *Expert Syst. Appl.* 287 (2025) 128089.
- [33] M.A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, H. Janicke, Edge-IIoTset: a new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning, *IEEE Access* 10 (2022) 40281–40306. <https://doi.org/10.1109/ACCESS.2022.3165809>
- [34] I. Ullah, Q.H. Mahmoud, A scheme for generating a dataset for anomalous activity detection in iot networks, in: C. Goutte, X. Zhu (Eds.), *Advances in Artificial Intelligence*, Springer International Publishing, Cham, 2020, pp. 508–520.
- [35] M. Sundararajan, A. Taly, Q. Yan, Axiomatic attribution for deep networks, in: *International Conference on Machine Learning*, PMLR, 2017, pp. 3319–3328.
- [36] H. Li, Q. Dong, J. Chen, H. Su, Y. Zhou, Q. Ai, Z. Ye, Y. Liu, LLMs-as-judges: a comprehensive survey on LLM-based evaluation methods, 2024, <https://arxiv.org/abs/2412.05579>.
- [37] M. Gao, X. Hu, X. Yin, J. Ruan, X. Pu, X. Wan, LLM-based NLG evaluation: current status and challenges, *Comput. Linguist.* 51 (2) (2025) 661–687. https://direct.mit.edu/coli/article-pdf/51/2/661/2513169/coli_a_00561.pdf https://doi.org/10.1162/coli_a_00561
- [38] C.-W. Liu, R. Lowe, I.V. Serban, M. Noseworthy, L. Charlin, J. Pineau, How not to evaluate your dialogue system: an empirical study of unsupervised evaluation metrics for dialogue response generation, in: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016, pp. 2122–2132.
- [39] E. Reiter, A structured review of the validity of BLEU, *Comput. Linguist.* 44 (3) (2018) 393–401. <https://aclanthology.org/J18-3002/>. https://doi.org/10.1162/coli_a_00322
- [40] A. Panickssery, S.R. Bowman, S. Feng, LLM evaluators recognize and favor their own generations, *Adv. Neural Inf. Process. Syst.* 37 (2024) 68772–68802. <https://doi.org/10.52202/079017-2197>